Migration of the Endpoint Management Infrastructure from Matrix42 Empirum to the baramundi Management Suite (bMS)

IT Project Report submitted for the professional qualification
IT Specialist for System Integration (IHK Baden-Württemberg)

Candidate: Vladimir Leicht
Training provider: TASys GmbH
Placement organisation: Enzkreis District Administration / Office for IT and Digitalisation
/ Pforzheim

Date: May 2026

Contents

1. Introduction.....4

1.1 Initial situation.....4

1.2 Analysis of the existing Empirum packaging.....4

1.3 Project objective and scope.....6

1.4 Economic aspects of the migration.....7

2. Technical prerequisites and foundations.....8

2.1 Provisioning the server VM.....8

2.2 Administrative and network foundations.....9

2.3 Server installation and initial configuration.....12

3. Installation of the baramundi Management Suite.....13

3.1 Setting up baramundi management accounts.....13

3.2 Installing the Management Suite.....17

3.2.1 Proxy connection for the bMS server.....17

3.3 DIP installation and configuration.....19

3.4 Defining a logical test client group.....21

3.5 Server management and initial job creation / assignment.....22

3.5.1 Self-management of the baramundi server.....25

3.6 PXE and TFTP services.....26

3.7 Preparing Windows 11 deployment.....26

3.7.1 Integrating the Windows 11 installation media.....26

3.8 Creating the WinPE boot environment.....27

3.9 Creating a baseline software bundle.....29

3.9.1 Baseline client software.....29

3.10 Driver integration.....31

4. Test rollout and troubleshooting.....33

4.1 Test rollout and fault analysis.....33

5. Project completion.....37

5.1 Planned versus actual results.....37

5.2 Time management.....38

5.3 Conclusion and outlook.....39

5.3.1 Conclusion.....39

5.3.2 Outlook.....39

5.4 Lessons learned / reflections.....40

Appendix A: Glossary and sources.....42

1. Introduction

1.1 Initial situation

This project report concerns the migration of the existing endpoint management infrastructure from Matrix42 Empirum[1] to the baramundi Management Suite[2]. The project was carried out at the Enzkreis District Administration in Pforzheim, specifically within the Office for IT and Digitalisation. Its aim was to develop a pilot environment that would provide the basis for the subsequent permanent introduction of a more stable endpoint management system. The starting point was the existing use of Empirum to manage Windows clients, software packages, drivers and operating system installations. Although the solution broadly fulfils the requirements of client management, operational experience has shown it to require substantial maintenance, be prone to errors and offer only limited scalability.

1.2 Analysis of the existing Empirum packaging

A significant drawback of the existing environment is its packaging process and usability. In practice, the existing Empirum packages rely heavily on Setup.inf and Empirum script structures. The existing 7-Zip package illustrates how even a comparatively simple MSI package contains numerous system-specific sections, variables, registry checks, MsiExec calls, log file evaluations and custom error-handling routines. Typical constructs such as DoesRegKeyExist, DoesTextInFileExist, RebootRequired and AbortMSIInst demonstrate that the package logic extends beyond a single installation parameter and must be maintained using a proprietary, product-specific syntax. An abbreviated example of an Empirum installation script for ‘7-Zip 24.01’ is shown below; the original file contains 203 lines.

```
##############################################

[SetupInfo]

Author = Adm_zensiert ; Package author

CreationDate = 13.02.2024 ; Package creation date

InventoryID = {B488FC15-B28F-4B63-A7D5-C9BD3707D009} ; Unique package ID

Description = Igor Pavlov 7-Zip 24.01 ; Package description

Method = MSI ; Installation method: MSI

Tested on = ; Tested systems could be listed here

Dependencies = ; Dependencies[14] on other packages

Command line options = /S2 ; Additional setup launch parameters
```

Last Change = 25.07.2023 ; Last change to the package

Build = 0 ; Package build revision

[VarDefInfo]

; <Variable>, <Type: 0=Computer 1=User>, <Allow NULL values: 0=No 1=Yes>

; Definition of user-specific or computer-specific variables

...

[RepairMSI]

AddMeter -1

; Enable the progress indicator

Call MsiExec /Fvomus "%SRC%\\%VM_InstFile%" ALLUSERS="1" REBOOT=REALLYSUPPRESS ARPSYSTEMCOMPONENT=1 /qb-! /Li "%MSILogFile%"

; Repair MSI / check all important components / write log

If %ErrorLevel[15]% == "3010" Then "RebootRequired" EndIf

; 3010 means a restart is required

If DoesTextInFileExist ("%ReInstSuccessMessage1031%", "%MSILogFile%") == "0" & DoesTextInFileExist ("%ReInstSuccessMessage1031_1%", "%MSILogFile%") == "0" & DoesTextInFileExist ("%ReInstSuccessMessage1033%", "%MSILogFile%") == "0" & DoesTextInFileExist ("%ReInstSuccessMessage1033_1%", "%MSILogFile%") == "0" Then "AbortMSIInst" EndIf

; If the log contains no success message, treat the installation as failed

Del "%MSILogFile%"

; Delete the log file

.

.

.

[InstallMSI]

AddMeter -1

; Enable the progress indicator

Call MsiExec /I "%SRC%\\%VM_InstFile%" ALLUSERS="1" REBOOT=REALLYSUPPRESS ARPSYSTEMCOMPONENT=1 /qb-! /Li "%MSILogFile%"

; Standard MSI installation in silent mode

If %ErrorLevel% == "3010" Then "RebootRequired" EndIf

; Handle the restart request
##

The usability disadvantage is clear. Administrators without in-depth Empirum experience find it difficult to understand changes to software packages, as installation logic, return values, registry detection and uninstallation behaviour are defined across multiple script sections. Maintaining the District Administration's existing package estate therefore still depends on understanding and maintaining Empirum scripts that have evolved over time.

There is also an organisational risk: at present, only two of the fifteen employees can 'reliably' create, assign and deploy packages using the existing packaging system. Owing to time constraints, this work is mainly performed by just one employee. Absence, familiarisation periods and concurrent requests have a direct impact on turnaround times for software deployment and client provisioning. The technical complexity of the scripts compounds this risk, as operational responsibility cannot be transferred to others at short notice.

Operational limitations also extend beyond packaging. The current PXE[7]-based operating system installation process is prone to failures, and patch management for third-party software does not always provide sufficiently transparent feedback. These factors create additional work when provisioning new clients and make reliable lifecycle management more difficult.

1.3 Project objective and scope

The project aims to introduce a baramundi pilot environment as a central unified endpoint management solution. Compared with Matrix42 Empirum, baramundi offers a more intuitive and accessible GUI, directly improving usability. The project scope covers setting up a bMS server connected to MS SQL, configuring Windows 11 deployment via PXE, integrating suitable driver profiles and deploying selected standard software. For the test run, the candidate selected 'Citrix Workspace App' (for access to virtualised desktops and applications), 'Cisco Webex', '7-Zip (x64)' (for communication, meetings and collaboration) and 'Adobe Acrobat Reader (x64)', together with required components such as 'Microsoft Visual C++ Redistributable' and 'Microsoft Edge WebView2 Runtime', as the baseline client software. This selection enables users to perform basic work and communicate on a newly installed client. Native packaging through bDeploy[5] is intended to make deployment easier to understand and maintain. This reduces dependence on legacy scripts that are difficult to maintain and improves accessibility for additional administrators.

The project's success is assessed through a pilot rollout to reference clients. The key criteria are functioning Windows 11 deployment, successful installation of the specified standard software, verifiable inventory collection and more reliable deployment feedback.

1.4 Economic aspects of the migration

From an economic perspective, the new solution is intended, among other things, to reduce administrative turnaround times and thus lower ongoing operating costs.

Publicly available manufacturer information describes the baramundi Management Suite as a modular UEM platform that can be operated on premises, in a hybrid configuration or as a hosted service. The SaaS offering is described as a model billed per client, with maintenance included. Recurring costs, maintenance charges, predictability and tied-up capital are therefore particularly relevant to the economic assessment. The specific amounts used in the assessment come from the net quotations provided to the candidate by the responsible officer within the organisation.

Over five years, the five-year subscription model is the least expensive option at €160,200.00 net. It is followed by purchase plus five years of maintenance at €175,715.00, a three-year subscription at €187,620.00, purchase plus three years of maintenance at €200,480.00, a one-year subscription at €221,280.00 and purchase plus one year of maintenance at €231,715.00. The difference from the least expensive purchase model is €15,515.00 net. The five-year subscription is therefore economically preferable for the defined project period; purchase models would need to be reconsidered only for a substantially longer period of use.

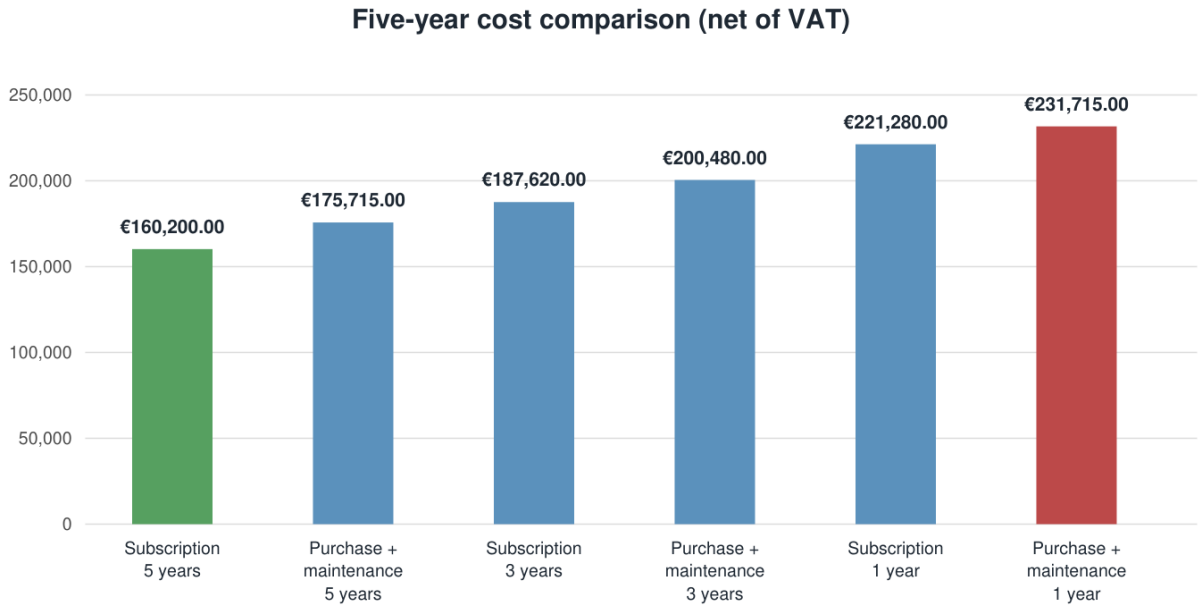


Chart 1: Comparison of quotation options over five years, net of VAT, in ascending order of total cost

Unfortunately, no specific financial comparison figures for Matrix42 Empirum can be provided within this project report. The responsible IT department employee, who joined the organisation in April 2026, does not have reliable data on this product. Given the breadth of the Matrix42 product range (https://marketplace.matrix42.com/de/?product-page=1), a complete assessment of all modules used within

the organisation and an accurate cost projection accounting for the managed endpoints would have exceeded the time available for this project by a considerable margin.

With around 1,300 endpoints currently supported and maintained by the Office for IT and Digitalisation, process costs, downtime and general administrative effort are often economically more significant than the licence price difference alone. A functioning IT infrastructure directly affects numerous District Administration employees and indirectly affects residents and businesses in the district. Licence costs are therefore not the primary reason for the planned migration; efficiency and operational reliability take priority. No extended decision matrix incorporating alternative software solutions was produced. The decision to use the baramundi Management Suite had already been made, as the responsible administrators had many years of positive experience with the product. Drawing specifically on this existing expertise eliminates time-consuming familiarisation and training phases entirely, shortening the project duration and substantially minimising implementation risk.

2. Technical prerequisites and foundations

2.1 Provisioning the server VM

The baramundi Management Server was provisioned as a virtual machine within the Enzkreis District Administration's existing VMware vSphere environment. The VM was first created in vCenter, assigned a unique hostname and placed in the designated application server folder. This integrated the server clearly into the existing structure and made subsequent administration unambiguous.

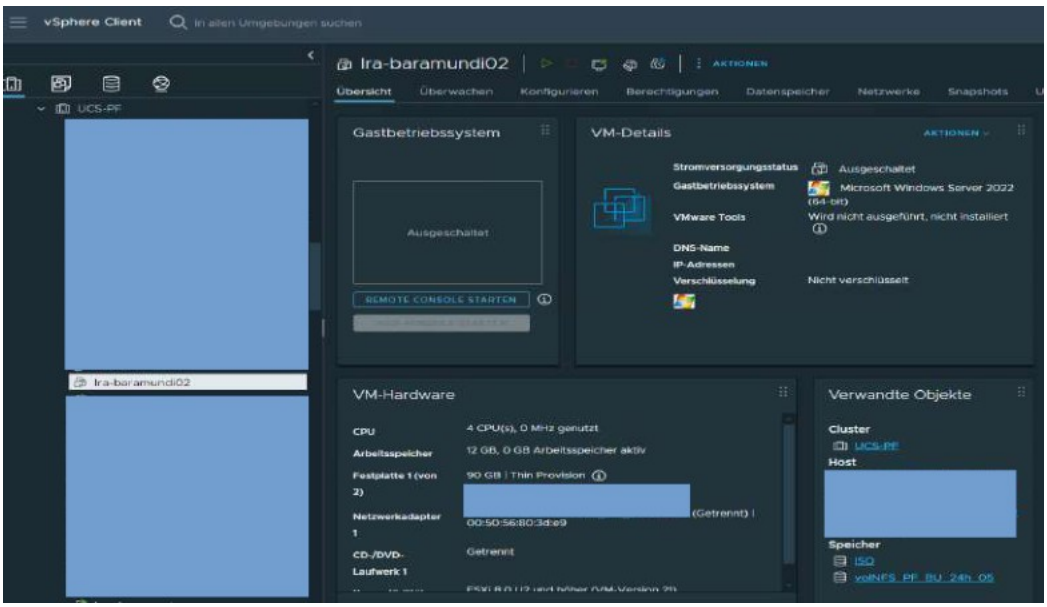


Figure 1: vSphere Client — creation of the virtual machine lra-baramundi02

The VM was allocated four vCPUs, 12 GB of RAM, a 90 GB system disk and an additional 500 GB data disk for the baramundi server's future DIP[3] directory, which would store software packages, drivers and operating system images. Separate disks were deliberately chosen to keep the operating system, applications and repository data logically separate and avoid resource bottlenecks on a shared drive. Connection to the production server VLAN was also considered at this stage, allowing the machine to operate in the intended network segment from the outset.

2.2 Administrative and network foundations

A dedicated security group was created in Active Directory for subsequent administration of the baramundi server. Following the AGDLP principle, this group was linked to the server's local Administrators group. Administrative privileges are consequently managed centrally through the domain structure rather than assigned directly to individual user accounts. Permission assignments remain traceable and role-based, and are easier to adjust following personnel changes.

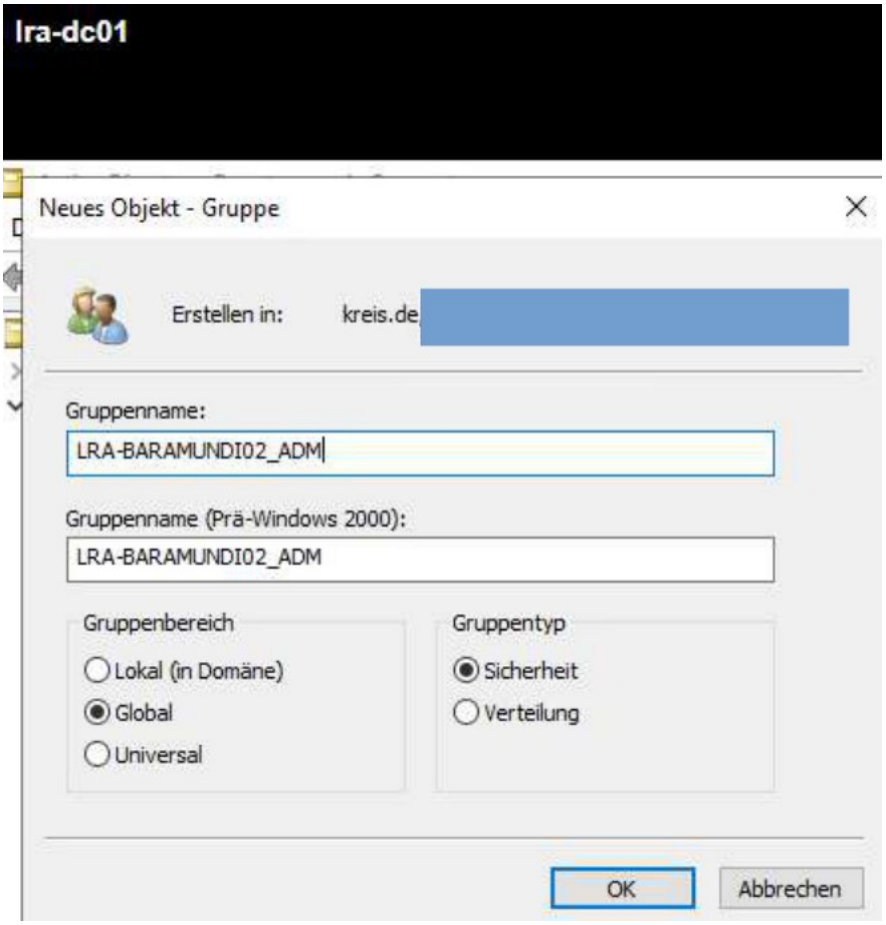


Figure 2: Creation of the AD administration group for the baramundi server

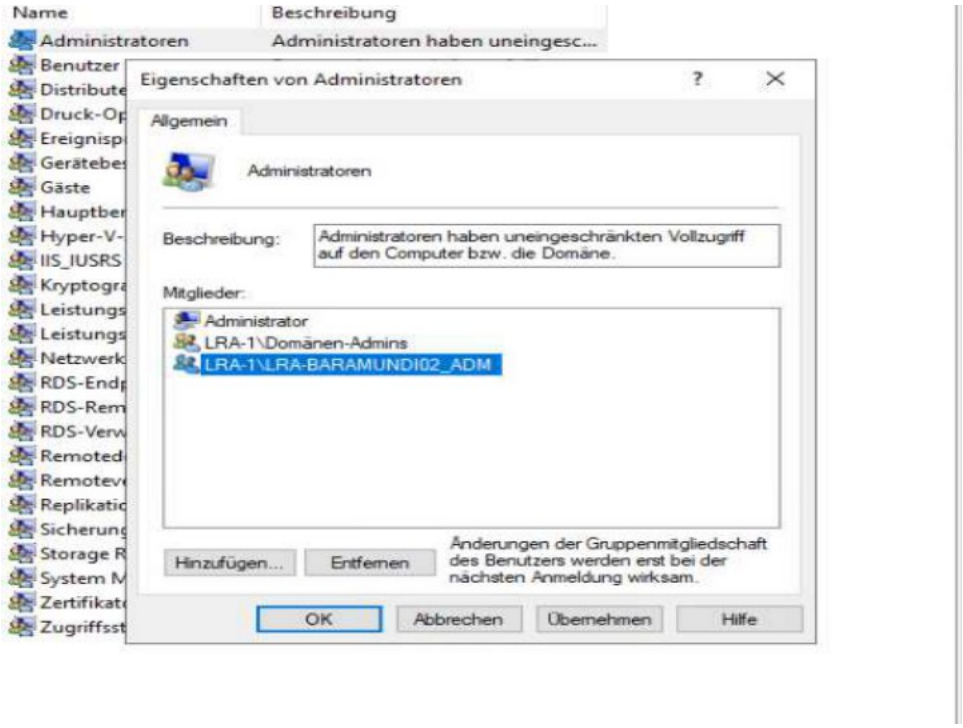


Figure 3: Windows Computer Management — nesting the new AD security group in the local Administrators group

Before installing and further configuring the baramundi server, the DHCP configuration in the designated test client scope had to be prepared for subsequent PXE booting of the reference clients. As the DHCP scopes form part of the central network infrastructure, which is particularly complex and sensitive, the technical changes were made by the responsible network administrators.

The boot parameters required for network booting were configured in the scope: option 66[24] points to the PXE/TFTP server at x.x.133.33, while option 67 specifies bootx64.efi[25] as the UEFI bootloader. During the DHCP exchange, clients therefore receive the server address and startup filename required to initiate PXE booting. For this purpose, the baramundi server was placed in the DMZ, in the same segment as the existing servers.

For the pilot, a separate VLAN segment (VLAN33) was provided at the designated RJ45 ports, where the target clients would subsequently be connected. Test clients could be connected specifically to these physical ports and automatically received an IP address from the designated address range through the configured DHCP service. Assignment through defined connection points made it easier to separate the test environment from the production client network, which had to remain active throughout the project, and ensured that DHCP, PXE and deployment tests remained verifiably restricted to the intended reference devices.

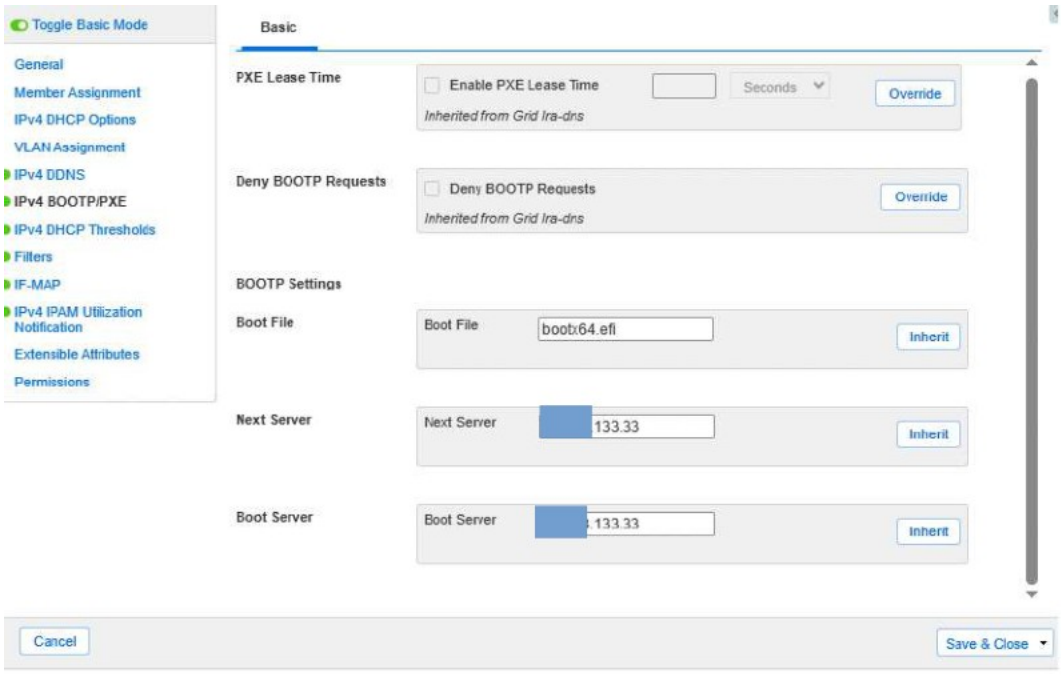


Figure 4: Infoblox IPAM/DHCP Manager — IPv4 DHCP boot parameters (option 66: x.x.133.33) in the test client scope

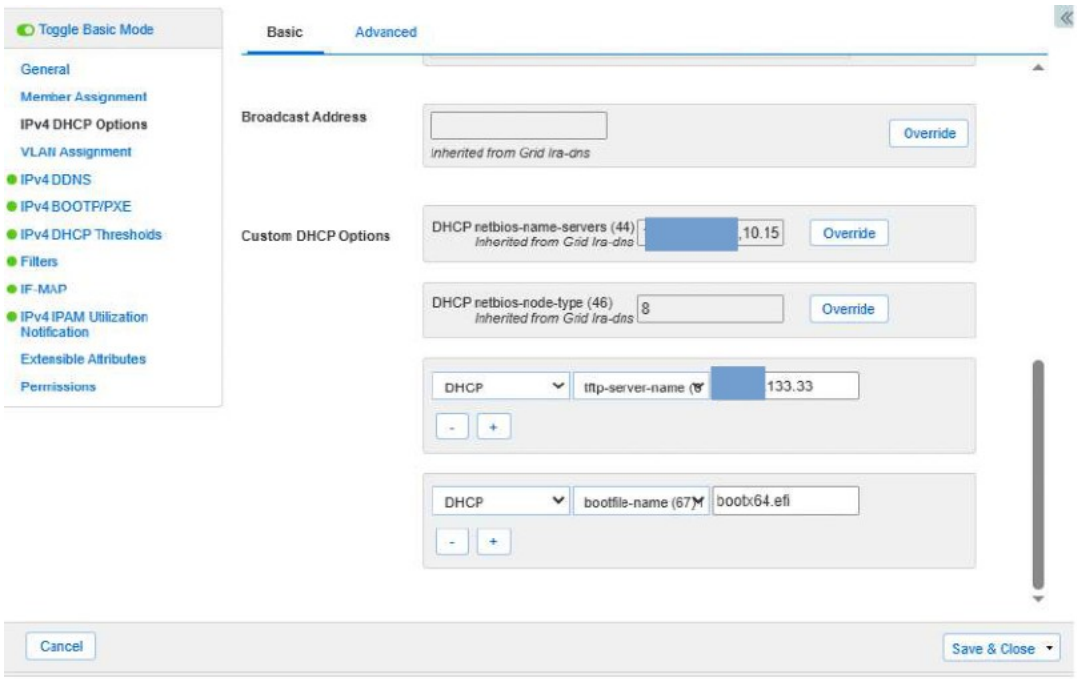


Figure 5: Infoblox IPAM/DHCP Manager — configuration of the UEFI bootloader (option 67: bootx64.efi)

As the server and target clients operated in different network segments, it was also necessary to check that routing permitted DHCP- and PXE-related requests to be forwarded. This coordination is essential for OS deployment: a correctly prepared boot image is insufficient if clients cannot reach the PXE service over the network. Following these changes, the foundations were in place for verifiable testing of reference client PXE booting through baramundi.

2.3 Server installation and initial configuration

Windows Server 2022 Datacenter was installed next, with a graphical interface because local GUI access was required for subsequent administration through the baramundi console. The IP configuration was then changed from DHCP to a static address, and DNS resolution was configured to use the domain's internal servers.

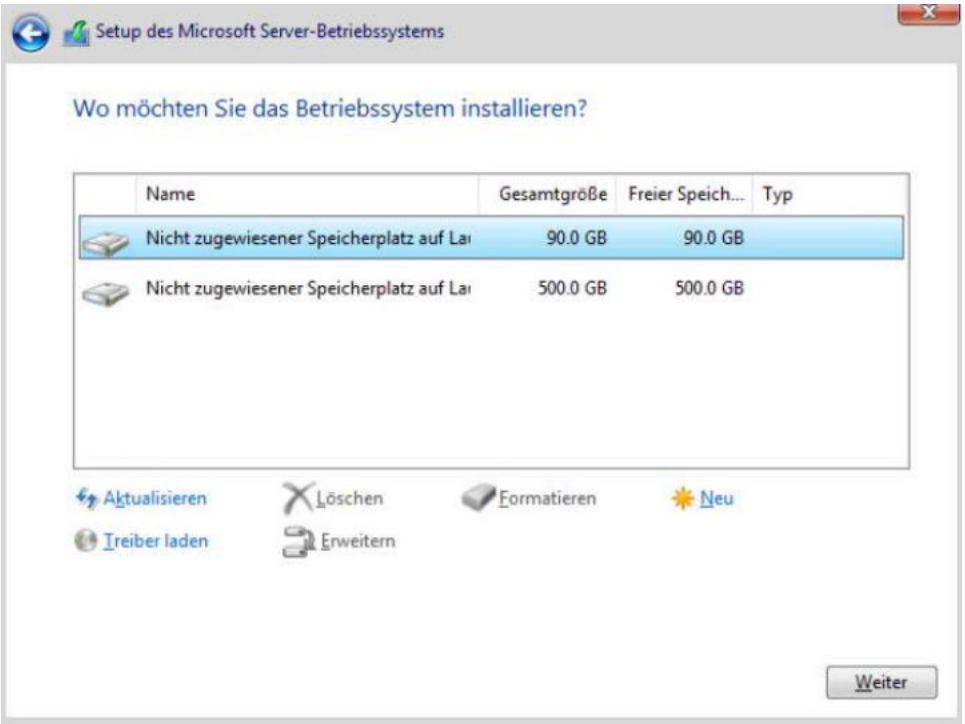


Figure 6: Installation of Windows Server 2022 on the VM's C partition

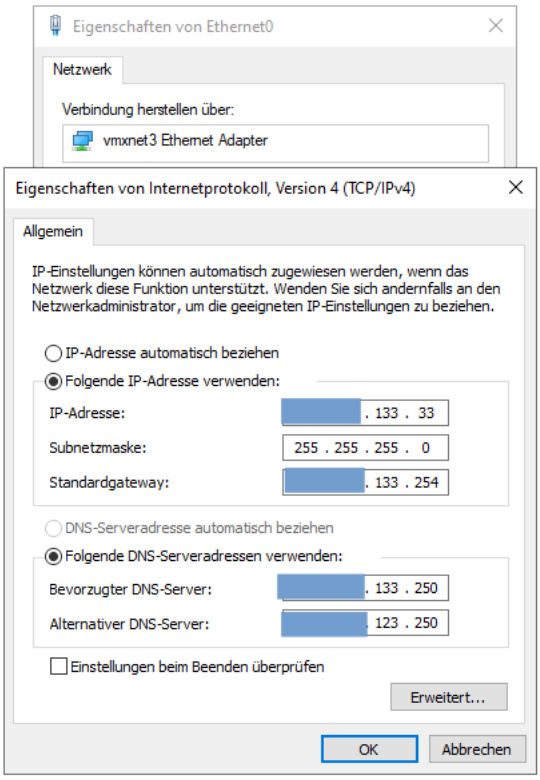


Figure 7: Static IPv4 configuration of the virtual network adapter (vmxnet3)

3. Installation of the baramundi Management Suite

Before the baramundi Management Suite itself could be installed, the required runtime environments had to be provisioned on the server. These included the Windows Desktop Runtime and the ASP.NET Core Runtime[13] in the appropriate 64-bit version. These components are required for stable operation of the graphical console and server-side services.

Figure 8: Provisioning the software dependencies (.NET Desktop Runtime and ASP.NET Core Runtime, 64-bit)

3.1 Setting up baramundi management accounts

Before the database could be configured and installation of the baramundi Management Suite (bMS) could begin, the necessary management and service accounts had to be prepared in Active Directory. Without these identities in place, successful installation and subsequent operation of the system are technically impossible. The baramundi architecture fundamentally relies on various background processes, such as the bMS server

service and PXE service, continuously performing administrative actions on the network, on target clients and within the SQL database. As these services can neither start nor communicate without valid security contexts, provisioning these accounts in advance is a critical system prerequisite.

Dedicated accounts serve two essential purposes: they enable the suite to function technically and ensure compliance with the principle of least privilege. Server actions, client access and network access are thereby kept strictly separate and isolated. The three core accounts defined for the project were baramundi_install, svc_baramundi and svc_baraNet.

As creating domain accounts and assigning network permissions fall within the remit of central IT security, a responsible member of the Infrastructure department created the accounts as requested and enabled them for the baramundi server environment.

- ‘baramundi_install’ (server and infrastructure account) was designated for tasks performed directly on the baramundi server. These include installing agents on Windows devices, storing files retrieved by download jobs, accessing MSW and patch sources[16] on the primary DIP, providing data for database import in the FileImport directory, reading BDS files[17], AD synchronisation and creating computer accounts during operating system installations.
- ‘svc_baramundi’ (installation user) was used as the installation user for server-side actions that must run within a defined user context. The account performs these actions on the server, accesses DIP$[18] for server-side processing and can be used to connect to Windows clients in the relevant job steps.
- ‘svc_baraNet’ (network user) was configured as the network user. This account is used when the baramundi Management Agent[8] needs to access network resources during job execution, for example in execution contexts such as LocalSystem, local installation user or installation user. It is also used to authenticate to baraDIP for bBT[9] transfers and DIP synchronisation.

The connection to the existing Microsoft SQL Server instance was then configured. Rather than creating a new local database environment, the database was to run in an existing, isolated SQL environment. A member of the Infrastructure department therefore created a dedicated SQL login for baramundi on the SQL Server.

To protect the isolated SQL environment in accordance with the principle of least privilege, the account was configured as follows:

No server administration privileges: the login has no server-wide administrative privileges (sysadmin), and the server roles remain at the default setting (public).

Database-level permission: the account is assigned only the db_owner role for the specific baramundi database.

This ensures that, during ongoing operation, the baramundi server service has all necessary read, write and schema permissions within its own database through the bCenter connection[19], without gaining access to other databases or the global server configuration.

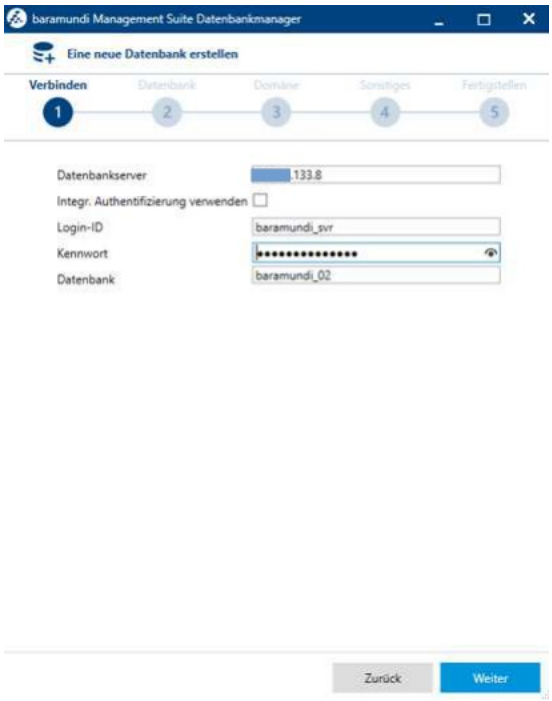


Figure 9: baramundi Database Manager — step 1: connection to MS SQL Server (x.x.133.8) using SQL authentication

The database connection was secured with TLS. The backend connection was thus fully prepared technically and could subsequently be initialised successfully.

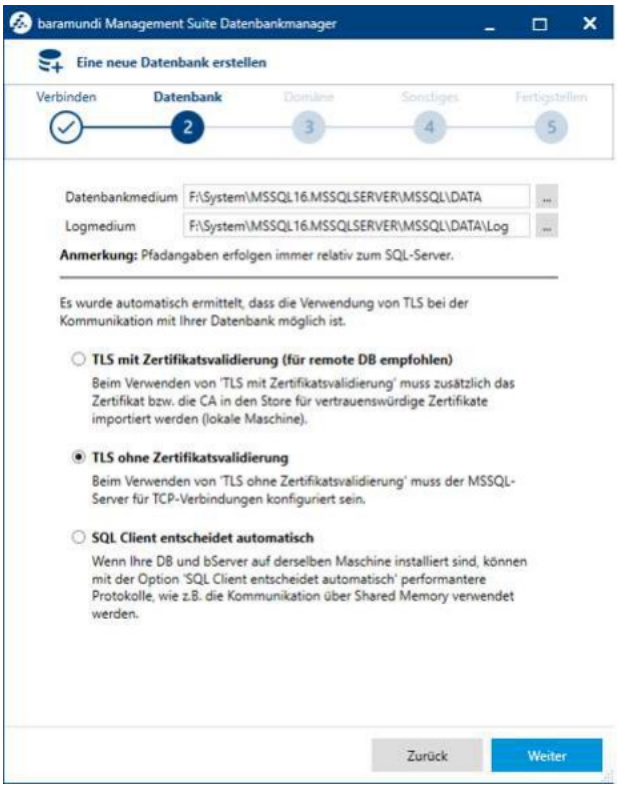


Figure 10: baramundi Database Manager — step 2: primary data (.mdf) and log (.ldf) paths with TLS encryption

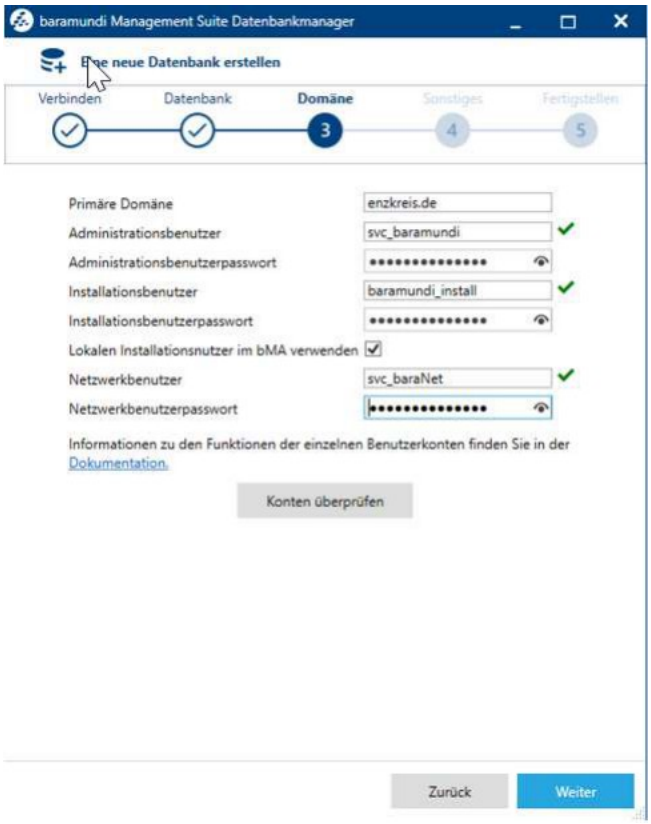


Figure 11: baramundi Database Manager — step 3: integration and validation of AD service accounts

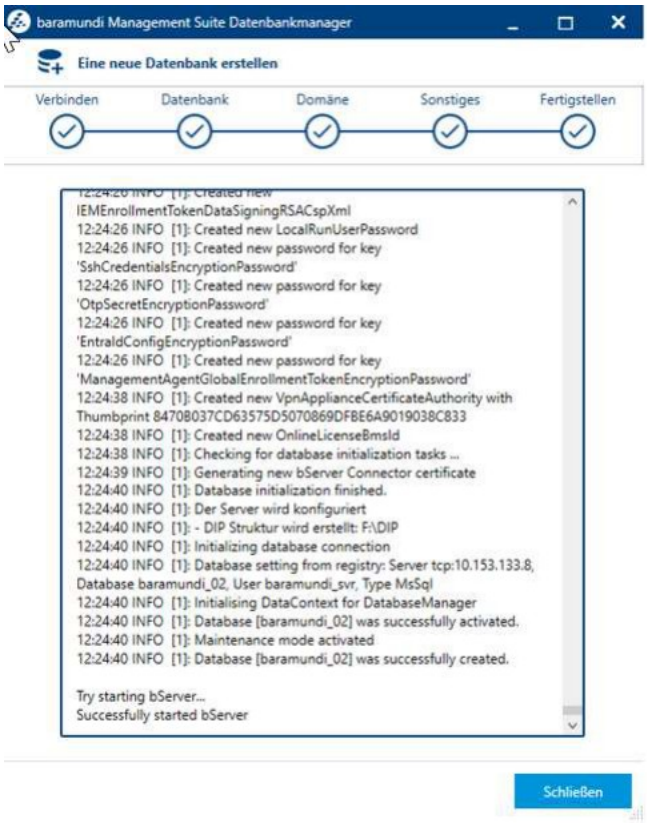


Figure 12: baramundi Database Manager — step 5: schema creation and cold start of the main server service

3.2 Installing the Management Suite

Following successful preparation of the server and database, baramundi Management Suite 2025 R2 was installed.

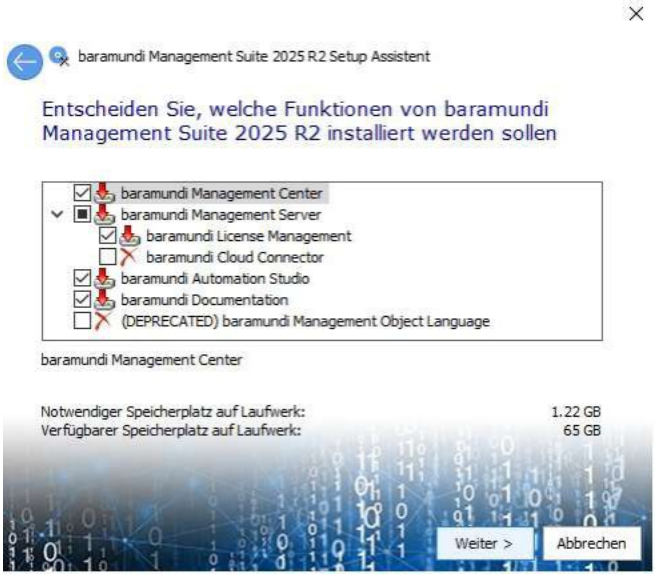


Figure 13: Selection of the baramundi Management Suite 2025 R2 components to install

3.2.1 Proxy connection for the bMS server

Immediately after installation, the candidate configured the proxy connection within the baramundi suite. This configuration is necessary for the baramundi Management Server to access external update and package sources through the District Administration's central infrastructure. During operation, it is required to ensure the continued availability of automated software updates and definition downloads through the dedicated proxy server.

Without documentation, administrators may find the proxy settings difficult to locate: the global internet connection is configured not centrally within network management, but several levels down under 'Konfiguration>Server>Einstellungen>Downloader' (Configuration > Server > Settings > Downloader).

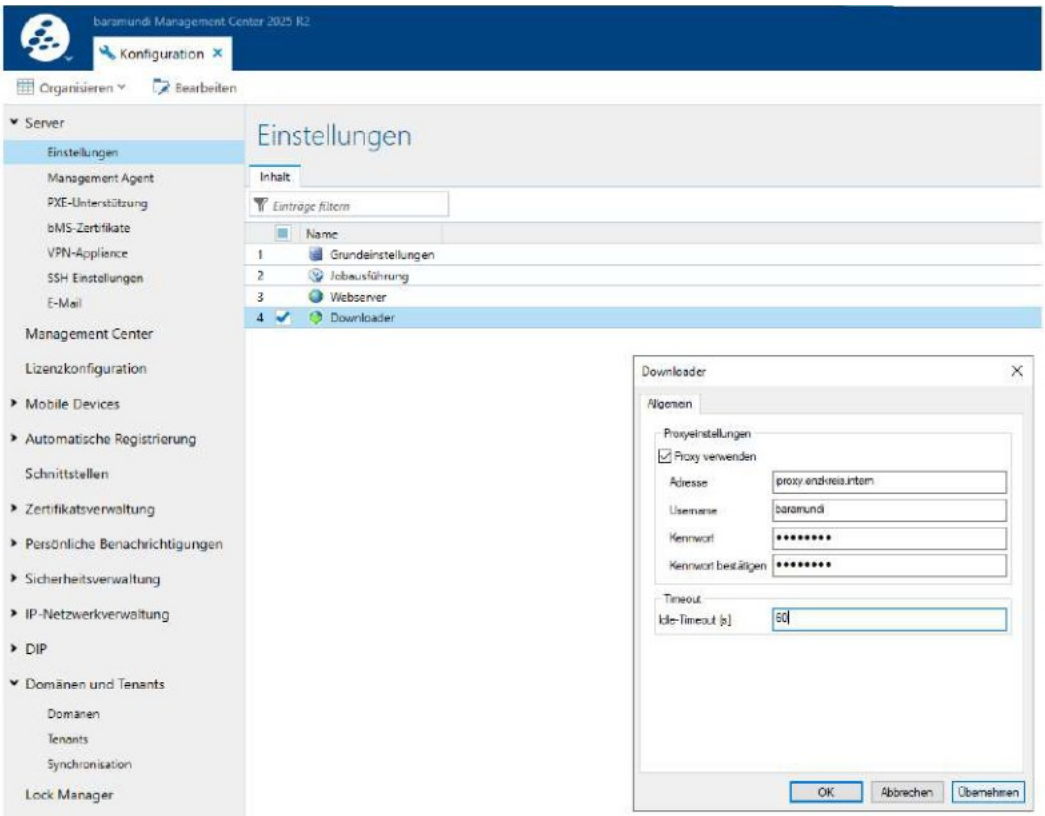


Figure 14: Configuration of the proxy connection using the credentials of the 'baramundi' account

Transport encryption between the server and clients was also enabled during installation. Communication uses HTTPS on port 443, providing additional protection for connections on the internal network. Once the services had started, the logical structure was configured in the Management Center so that subsequent jobs, groups and target systems could be represented clearly. The bConnect interface[4] was configured to provide secure, encrypted communication between the baramundi Management Server and external network components, such as the baramundi Gateway for clients used when working from home or external third-party systems (APIs). Communication is routed through the standard HTTPS port 443. Integrated Windows authentication is enabled to secure access and process only authorised requests within the domain structure. As this was a purely on-premises pilot, optional cloud connector services were deliberately left disabled at this stage.

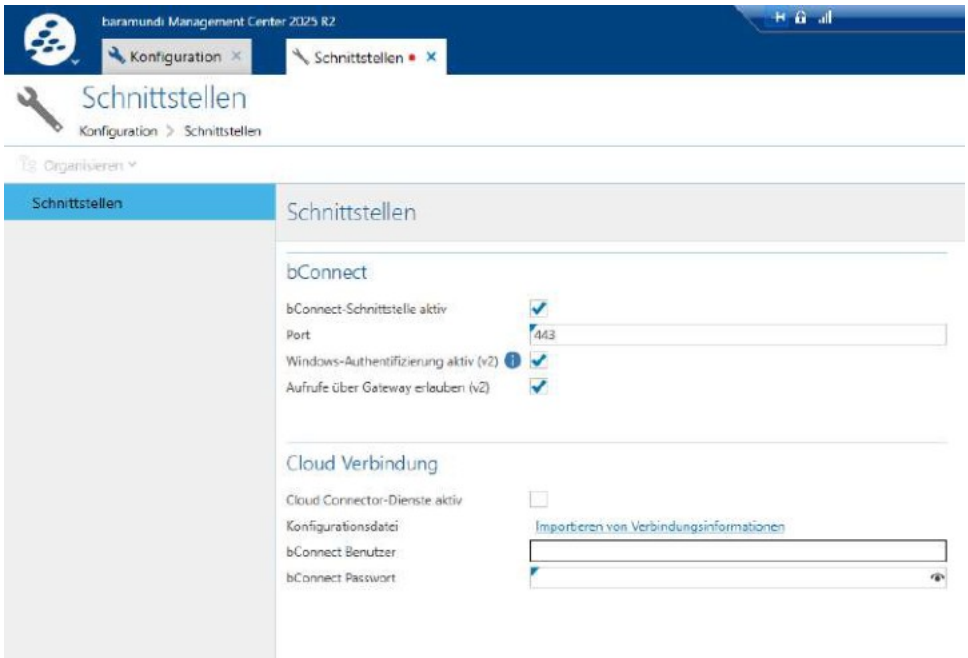


Figure 15: bMC interfaces — enabling the TLS-encrypted bConnect interface on port 443

3.3 DIP installation and configuration

The Distributed Installation Point (DIP) was the next logical component to install within the baramundi management architecture. It supports efficient delivery of software deployments, drivers and operating system installations in distributed IT environments. As the Distributed Installation Point runs on the same server as the baramundi Management Suite, its primary role in this environment is to provide installation sources and software packages. The advantages typically associated with a DIP at distributed sites, such as reducing WAN traffic or providing data locally in remote network segments, do not apply in this test environment. It was nevertheless implemented in accordance with baramundi's intended architecture, establishing a clear, scalable basis for later infrastructure expansion.

When setting up this component, the candidate was initially unable to establish a connection between the baramundi server and the Distributed Installation Point (DIP). Checking the baramundi configuration and reviewing the available documentation did not resolve the issue, so troubleshooting was extended to the server and network layers.

Following a more detailed analysis and consultation with the supervising specialist at the placement organisation, the candidate identified the cause in the network configuration of the virtualised Windows Server 2022. In dual-stack operation, IPv6 addresses were prioritised over IPv4 by default, whereas the existing baramundi infrastructure was primarily designed for IPv4 communication. This caused problems with the DIP connection.

To resolve the issue, the candidate adjusted the network protocol priorities so that IPv4 was preferred during address selection, without disabling IPv6. The following command was run in an administrative command prompt:

netsh interface ipv6 set prefixpolicy ::ffff:0:0/96 60 4

Adjusting the prefix policy ensured that IPv4 was preferred. The candidate was then able to connect to the DIP successfully and complete its configuration.

Integrating the DIP also required permissions to be configured on the corresponding DIP share. Access was granted to the relevant administrative users and the required baramundi service accounts. As described above, the accounts concerned were baramundi_install, svc_baramundi and svc_baraNet. These accounts were given read and write permissions on the DIP directory.

These permissions are required for the baramundi server, installation services and configured network user to store, update and read installation files, software packages and deployment data in the DIP directory. Without correctly assigned permissions, package transfers, DIP synchronisation and subsequent software deployments could not be performed reliably.

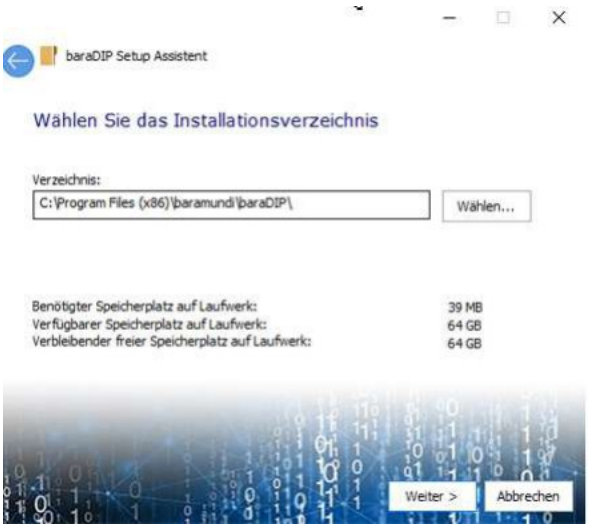


Figure 16: Installation of the DIP module required to manage the subsequently configured DIP server on partition F:

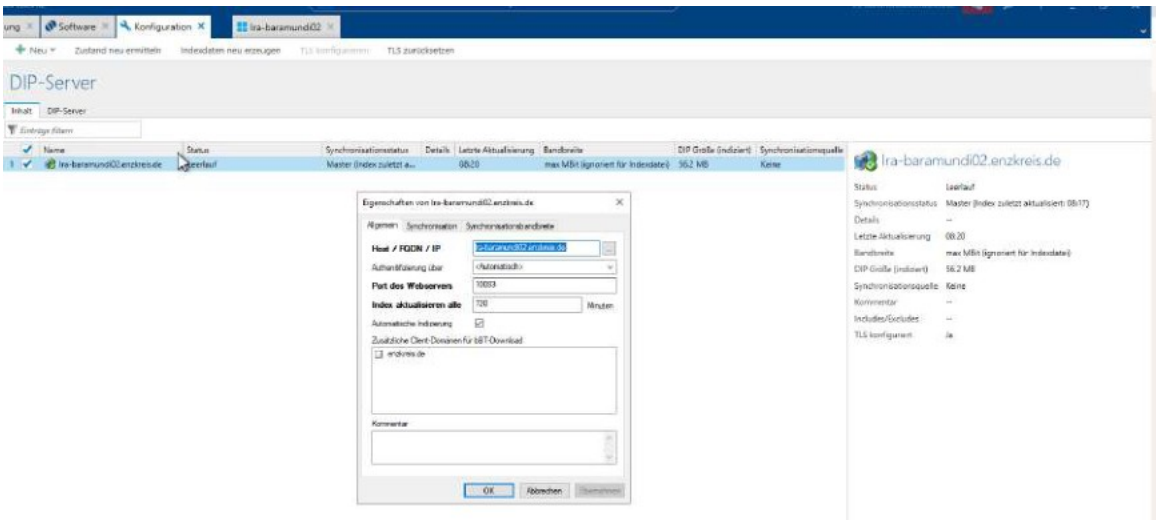


Figure 17: Creation and configuration of the DIP server

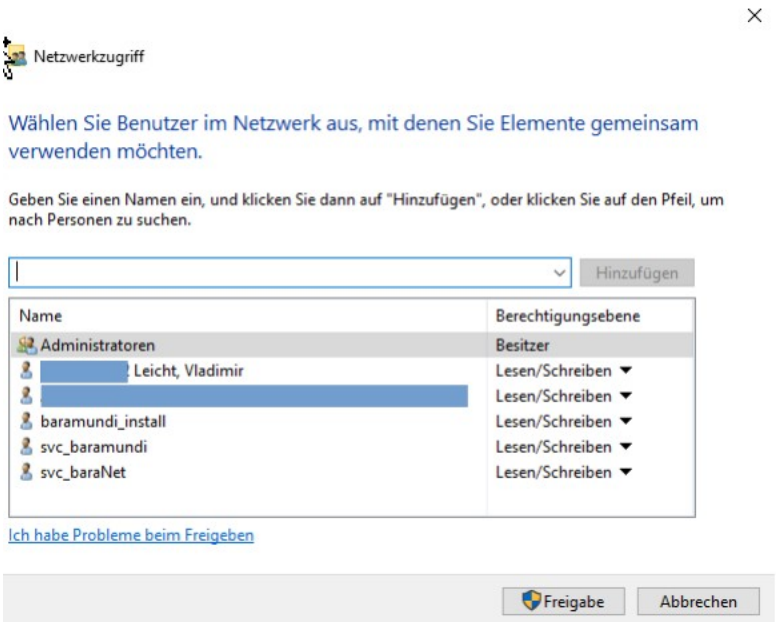


Figure 18: Account permission assignments for access to F:\DIP

3.4 Defining a logical test client group

The next step was to create suitable logical groups for both the managed clients (Testgruppe_UEM_Evaluation) and the baramundi server (enzkreis.de). Automatic assignment of new clients to 'Testgruppe_UEM_Evaluation' was also configured. As soon as they appear within the defined subnet scope and are directed to the baramundi server through DHCP, the Management Center automatically moves them into 'Testgruppe_UEM_Evaluation' and initiates installation of both the operating system and the assembled baseline software bundle ('Basis-Software-Clients').

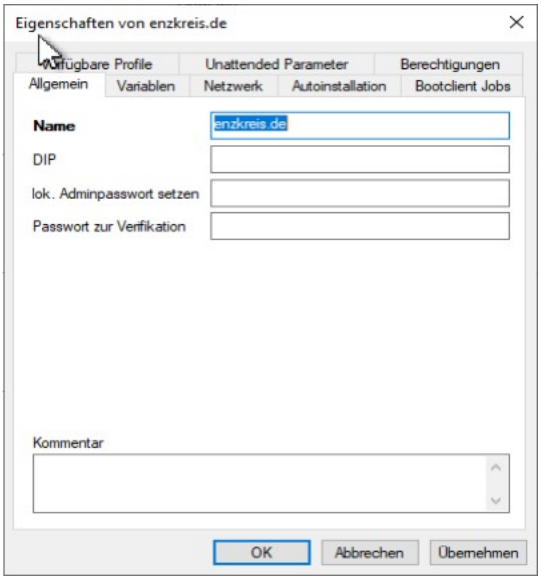


Figure 19: Creation of the logical group 'enzkreis.de' in the environment tree

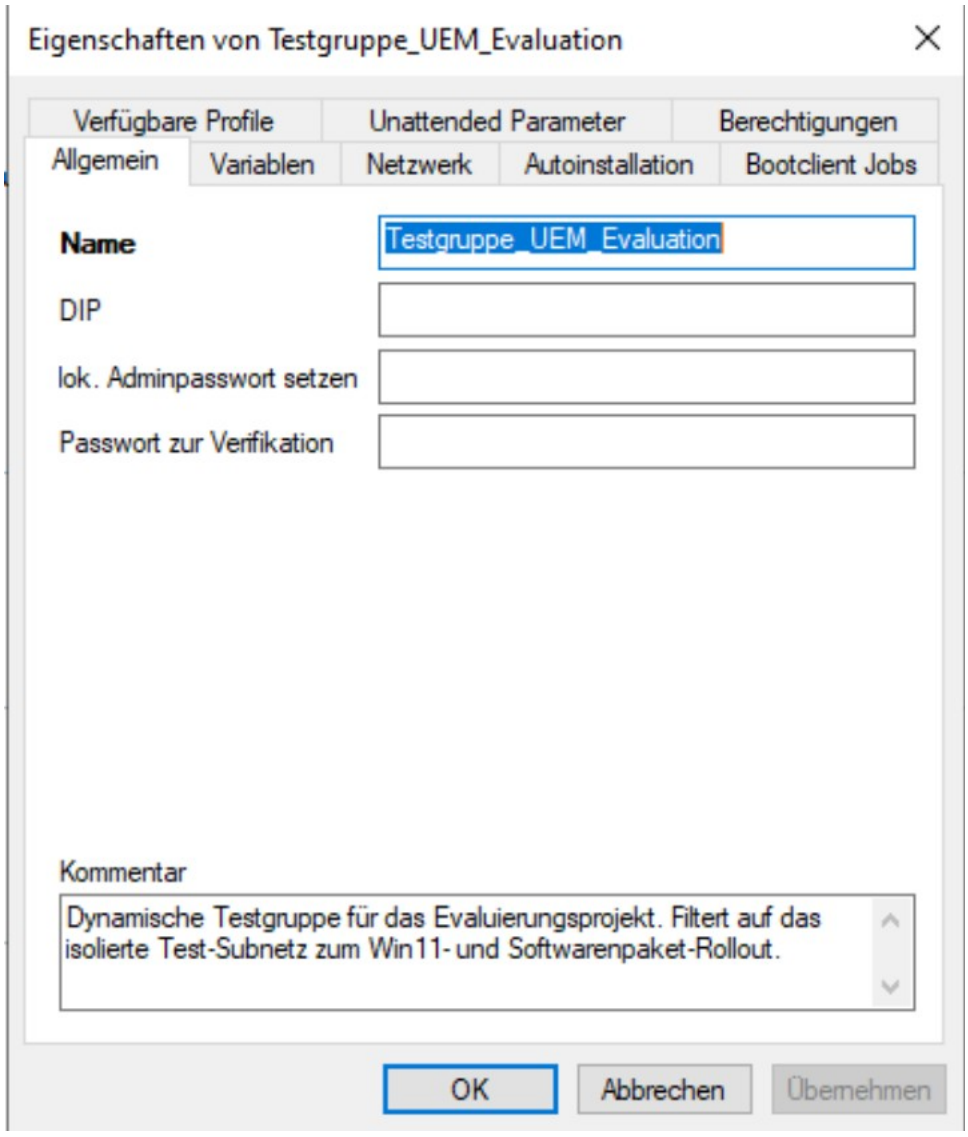


Figure 20: Creation of the logical group 'Testgruppe_UEM_Evaluation' in the environment tree

3.5 Server management and initial job creation / assignment

The best-practice approach is to install an agent on the baramundi server itself. This enables both software and hardware inventory collection for the server, as well as update management for its installed software and individual baramundi modules. The candidate first had to add the baramundi server itself to the baramundi environment using its IP address. The logical group 'enzkreis.de' was selected to keep it clearly separate from the subsequent clients, which were to be recorded in 'Testgruppe_UEM_Evaluation'. Once this was complete, an agent could be deployed to the baramundi server and a baramundi job named Inventur_HW_SW_WMI could be created. This job could also serve as a template for future inventory job assignments to clients.

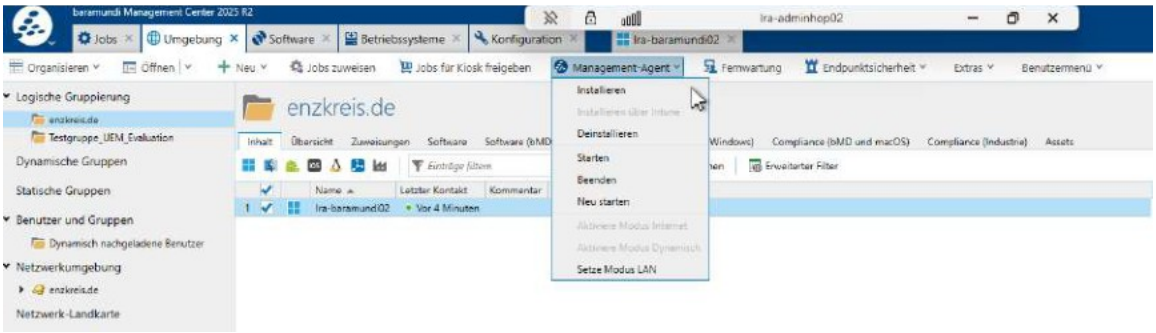


Figure 21: Logical grouping of the baramundi server and agent control in the Management Center

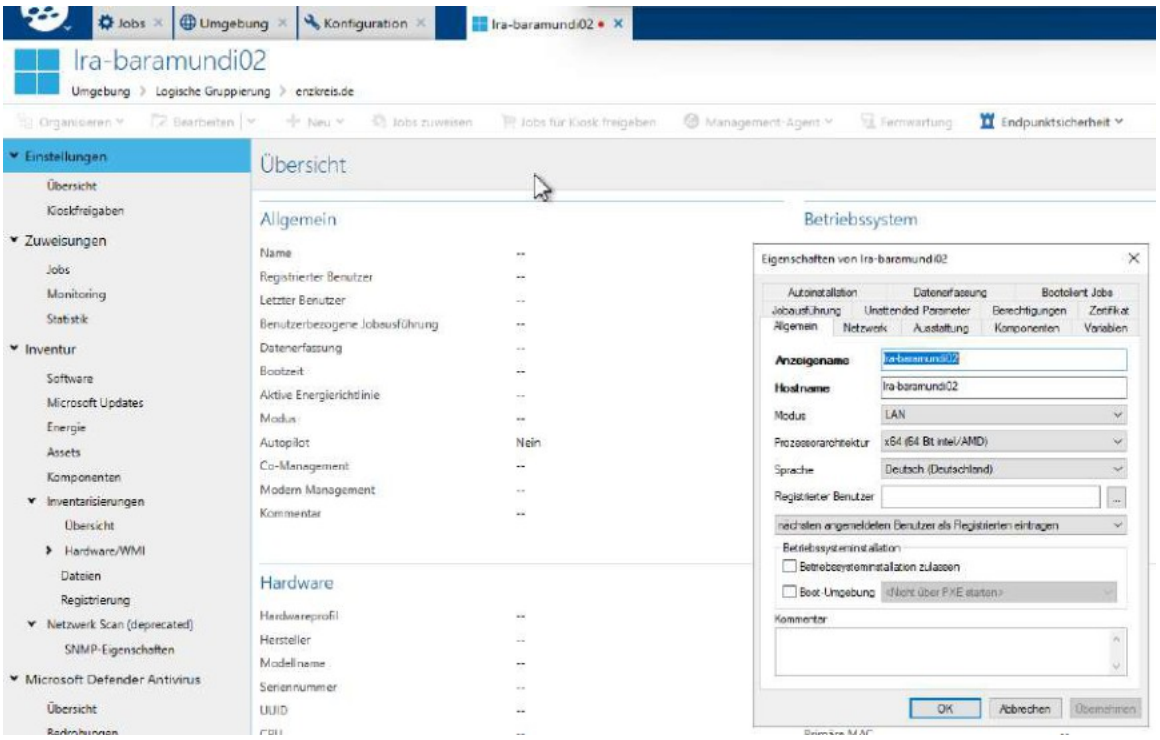


Figure 22: Adding the baramundi server to the Management Suite environment

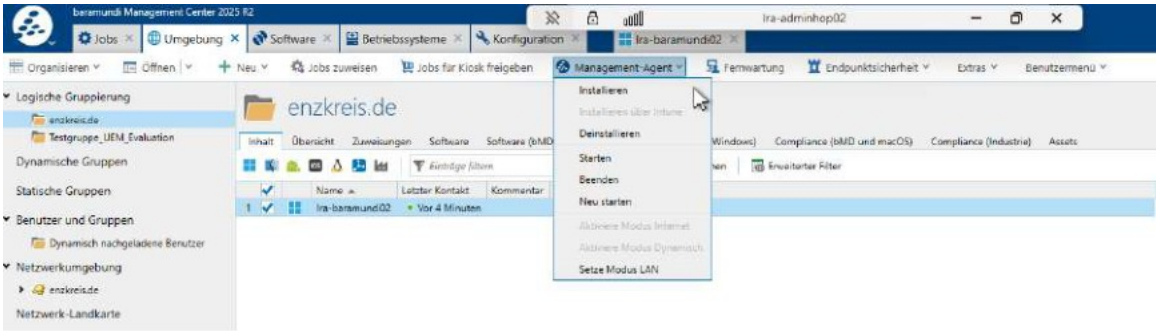


Figure 23: bMC — agent installation on the baramundi server for automated maintenance and inventory collection

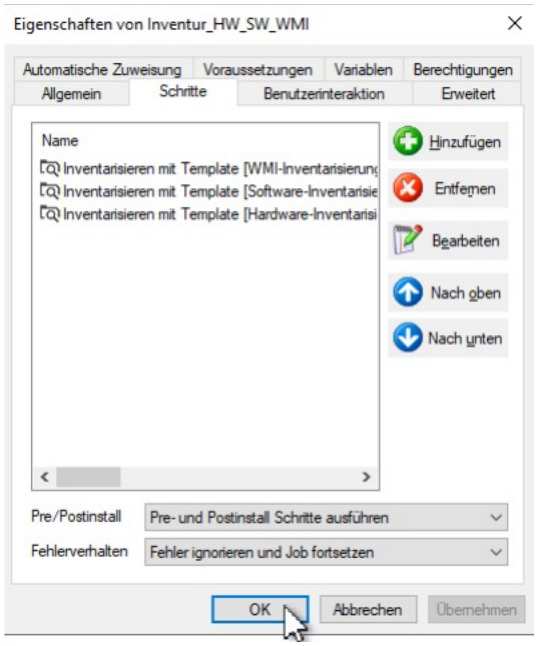


Figure 24: Creation of an inventory job

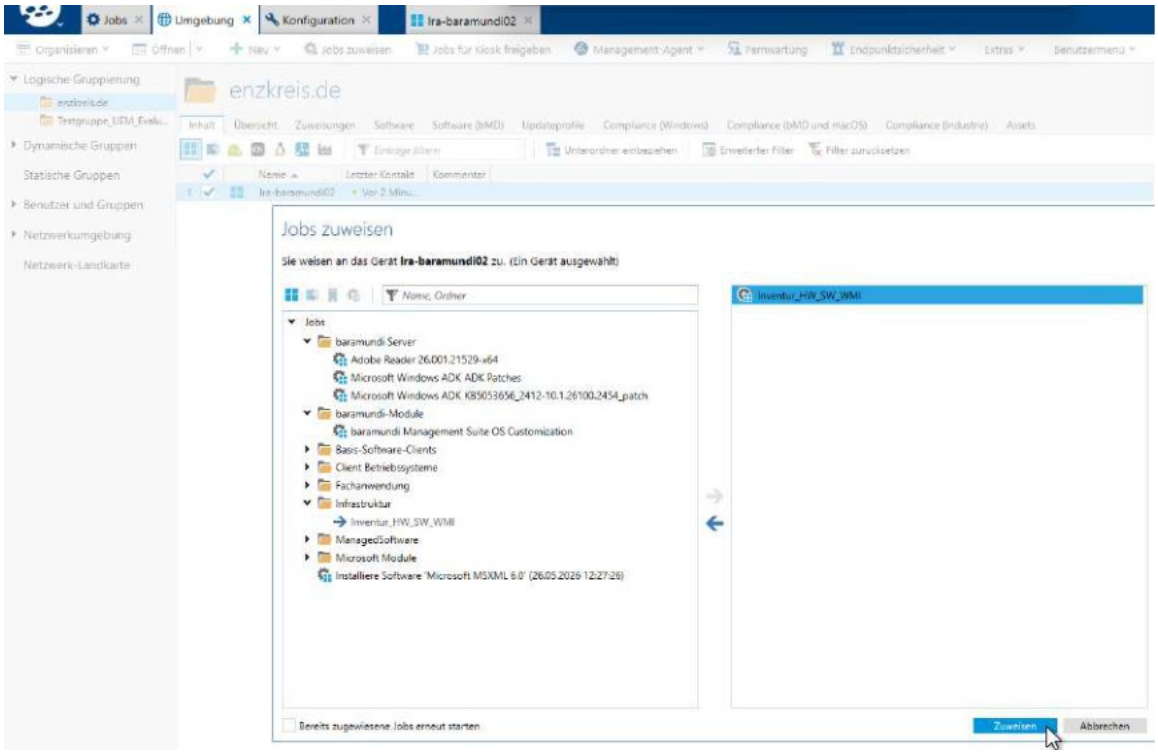


Figure 25: Assignment of the inventory job to the baramundi server

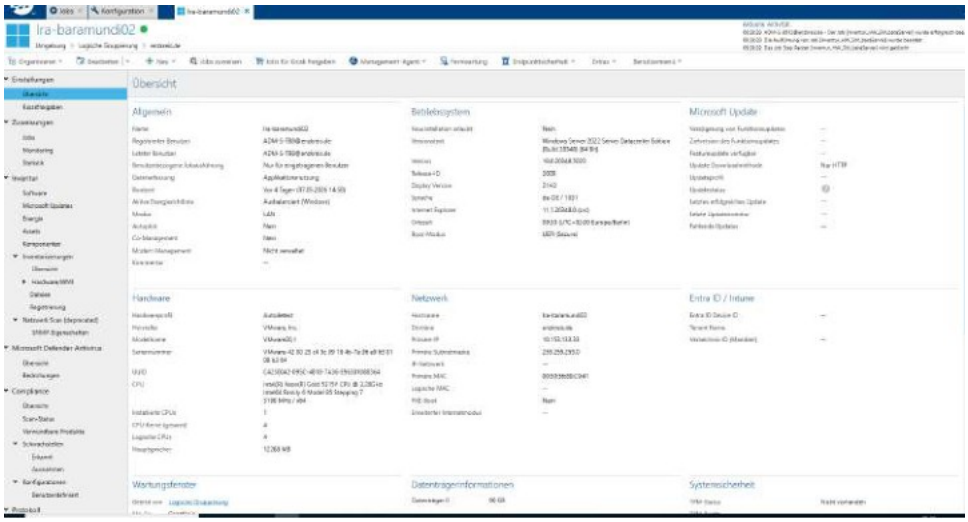


Figure 26: Hardware Inventory tab — first successful collection of physical component data

3.5.1 Self-management of the baramundi server

An additional infrastructure job was configured for self-management of the baramundi server. It combines software inventory collection, Windows and Microsoft updates, and Microsoft Defender definition updates. The suite can thus monitor the management server itself and keep its core components up to date without mixing these tasks with the client rollout jobs. The screenshot shows the successfully completed inventory job and the planned steps for update and definition maintenance.

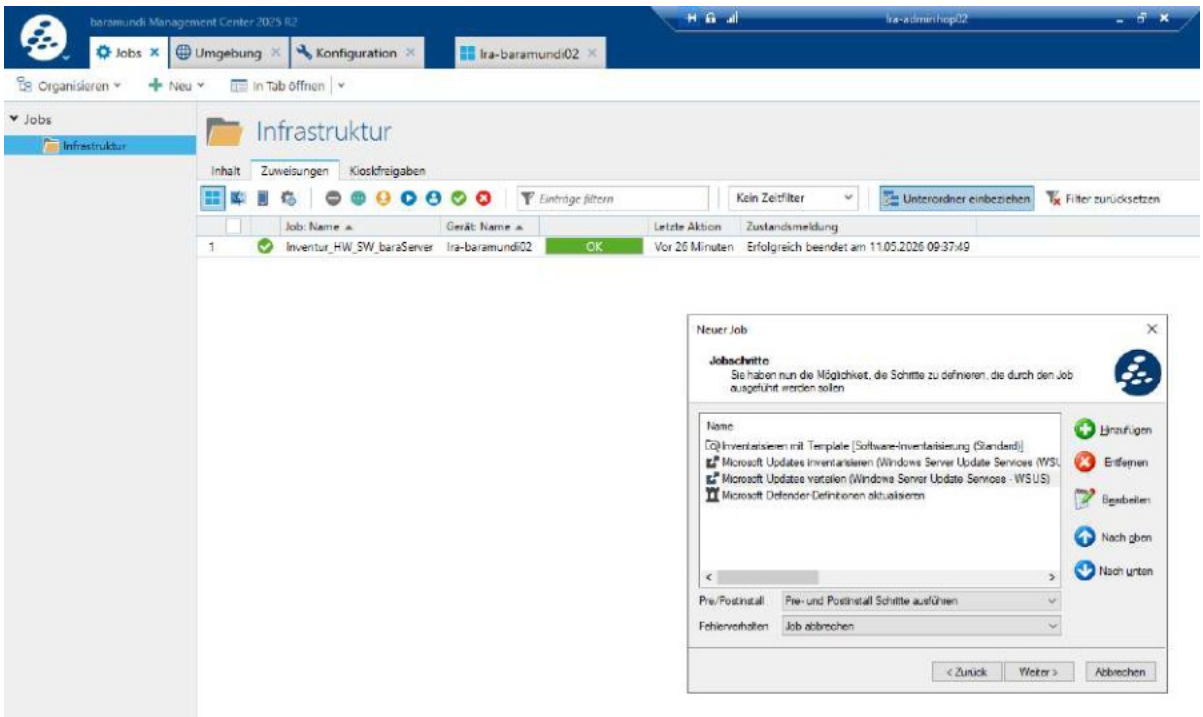


Figure 27: baramundi server self-management job with inventory collection and Microsoft updates

3.6 PXE and TFTP services

The PXE infrastructure was prepared for automated operating system deployment. This included enabling and configuring the baramundi PXE and TFTP services. The TFTP service provides the startup files, while PXE control itself was accounted for in the existing DHCP and network configuration.

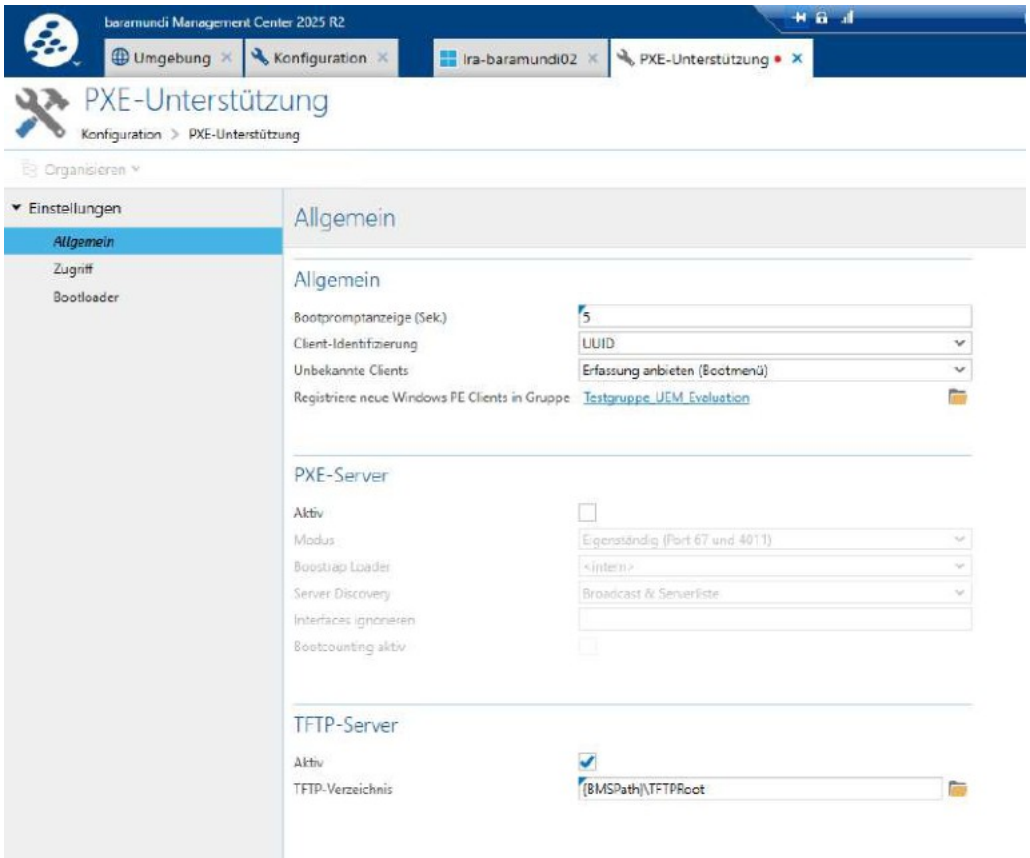


Figure 28: Configuration of the PXE and TFTP services in the baramundi Management Center

3.7 Preparing Windows 11 deployment

3.7.1 Integrating the Windows 11 installation media

The next step was to integrate the Windows 11 installation media natively into baramundi. The ISO file served as the central source for operating system deployment; its install.wim was used as the operating system image for the subsequent unattended installation. Installation parameters were also defined, including the edition, target partition, unattended answer file and handling of the computer account.

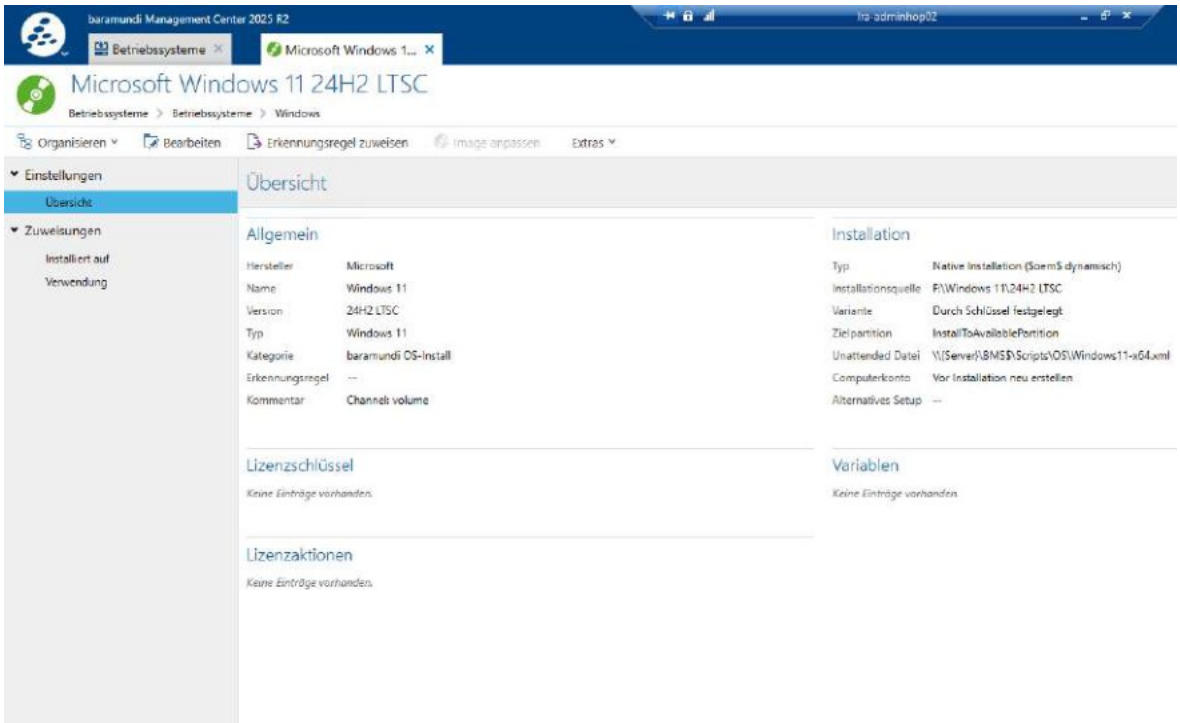


Figure 29: Native integration of the Windows installation media into baramundi

3.8 Creating the WinPE boot environment

A dedicated WinPE boot environment was required for PXE booting. Before it could be created, Microsoft Windows ADK[20], the WinPE add-on and the DISM components all had to be available, as these tools provide the technical basis for creating and customising the boot environment.

Only after the Windows installation media had been integrated was the Boot Media Wizard[6] used to create the PXE-capable WinPE environment. The architecture, UEFI boot mode, Windows Kit path, source WIM and target directory within TFTPRoot were specified. The wizard uses these settings to generate boot.wim, which is then distributed to the PXE/TFTP or deployment infrastructure and subsequently loaded by the client over PXE/TFTP.

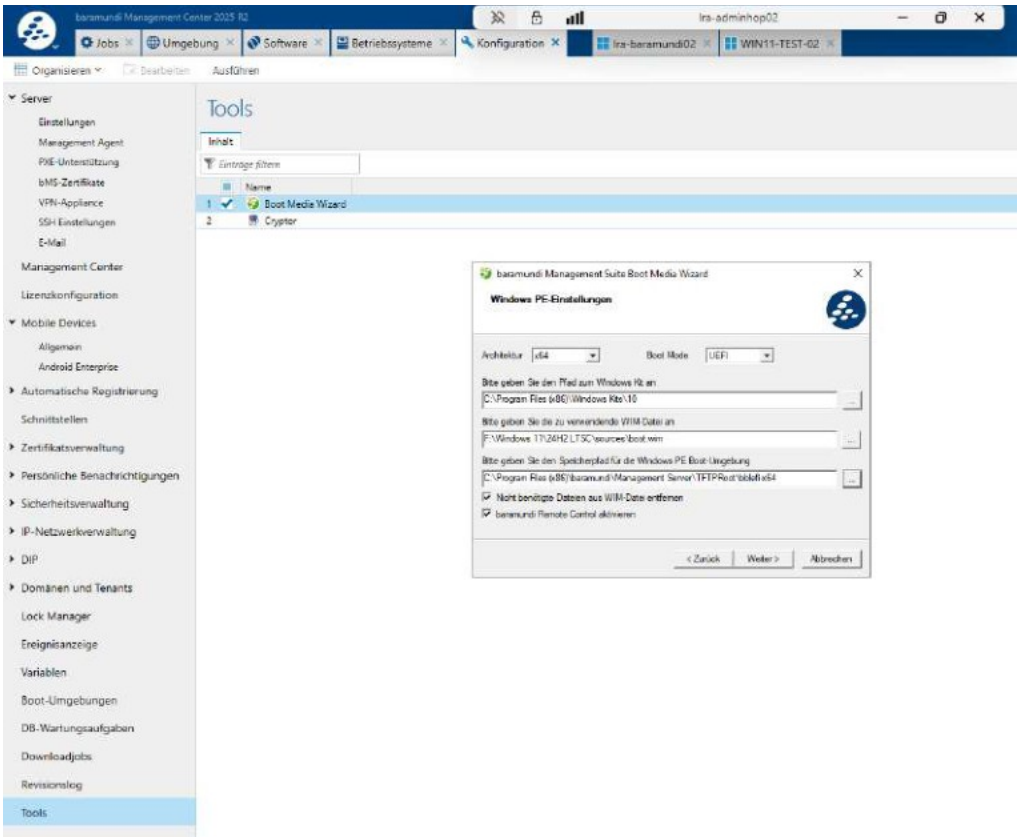


Figure 30: Boot Media Wizard — creation of the WinPE boot environment

A subsequent check of the file location confirmed that the generated boot.wim had been placed in the TFTP directory and was therefore available as a boot image for the PXE/TFTP infrastructure. This file is not the actual Windows installation image; it is the WinPE startup environment for network booting. Its size of approximately 427,986 KB is plausible for a WinPE boot.wim containing baramundi components and network and storage drivers.

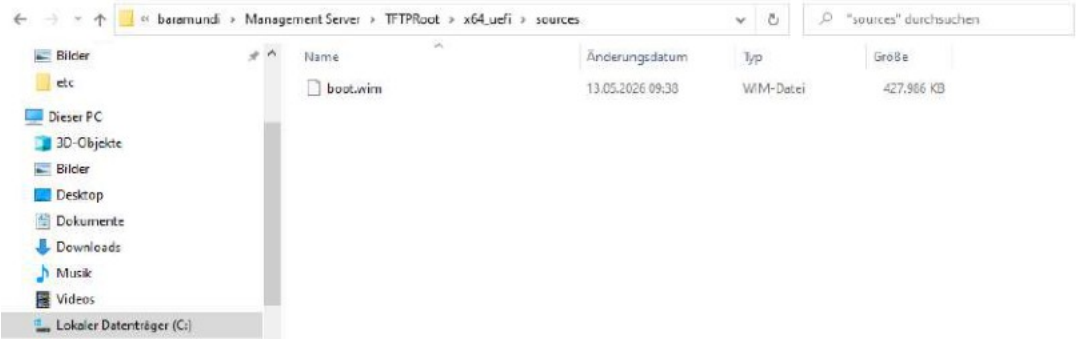


Figure 31: File system — validation of the generated boot.wim in the TFTPRoot directory

After integrating the installation media, creating the WinPE boot environment and making the boot image available, a dedicated installation job was created for the client operating systems. It groups the steps required for Windows 11 deployment into a controllable unit for later assignment to the test group or to reference clients.

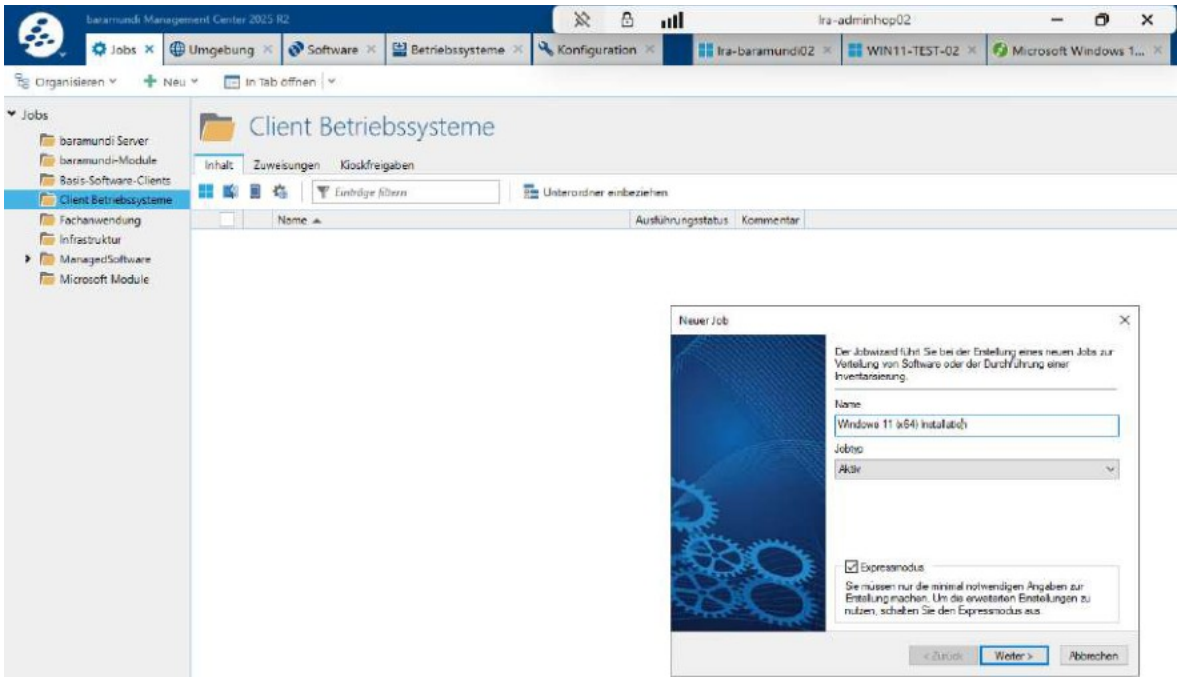


Figure 32: Creation of the Windows 11 installation job for client operating systems

3.9 Creating a baseline software bundle

3.9.1 Baseline client software

As part of baseline client provisioning, a dedicated baramundi job was created to install the standard software. The applications and dependencies required for testing were combined into a single installation step. The software packages were obtained through baramundi Managed Software, allowing them to be provided and updated centrally and used with the appropriate installation logic.

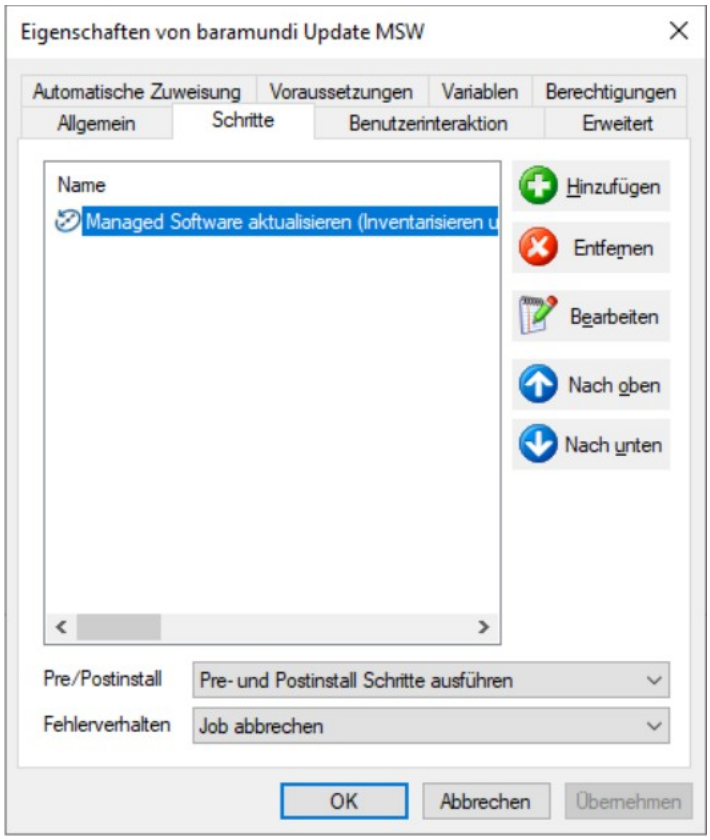


Figure 33: Creation of a Managed Software job with a defined installation step

The resulting Basis_Software_Clients_Job includes Microsoft Visual C++ Redistributable, Microsoft Edge WebView2 Runtime, Citrix Workspace App, Cisco Webex, Adobe Reader DC and 7-Zip, among other components. These components were deliberately provided through a subsequent software job rather than integrated into the Windows image.

The Windows 11 image and baseline software package were built as separate modules. This keeps the operating system image lean and easy to maintain, while applications can be updated, replaced or extended independently. Software changes therefore do not require the image to be recreated and can instead be controlled through baramundi jobs.

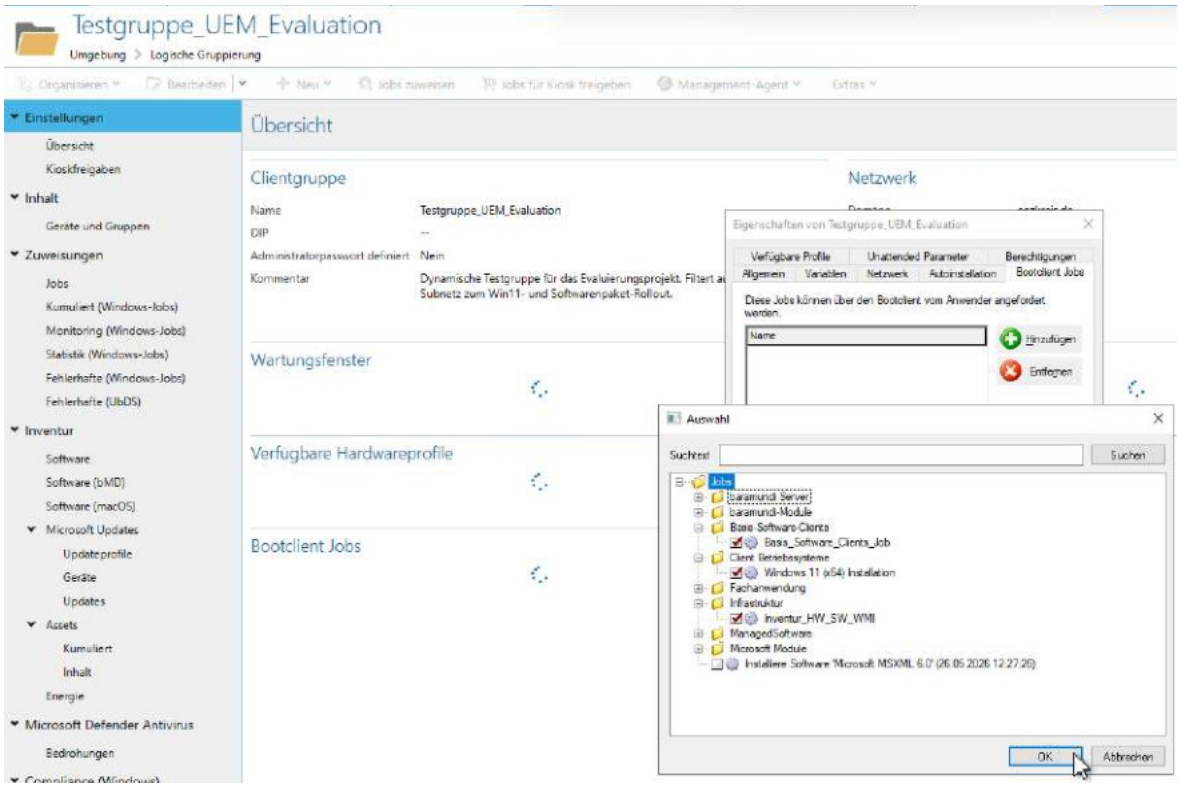


Figure 34: Assignment of the created jobs to Testgruppe_UEM_Evaluation

Once the OS installation job[11], baseline software bundle and inventory job had been created, they could be assigned to the test group. This group assignment formed the basis for the subsequent rollout to the intended reference clients.

3.10 Driver integration

Driver integration distinguished between two use cases. The WinPE boot environment requires network and storage drivers in particular, enabling the client to initialise its hardware after PXE booting and reach the baramundi server or DIP. When these drivers are integrated directly into boot.wim using the Boot Media Wizard, no separate driver profile is required.

Drivers for the subsequently installed Windows system must be considered separately. Model-specific driver profiles were prepared for this post-installation stage. They allow suitable drivers to be assigned to a particular hardware model or OS installation job and delivered in a controlled manner during deployment.

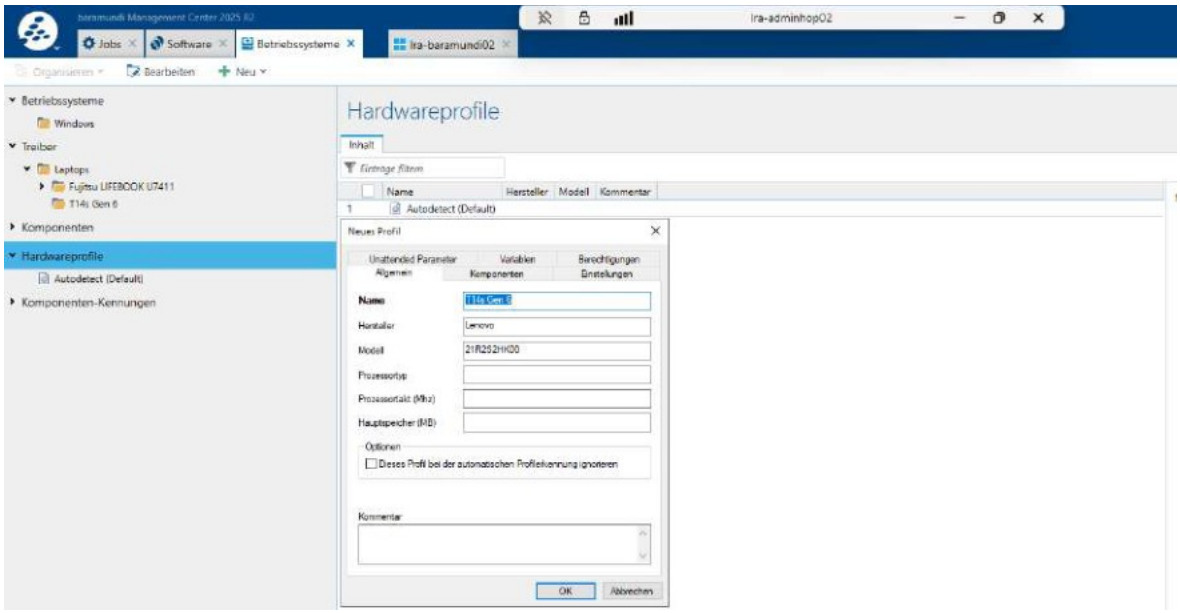


Figure 35: Creation of a hardware profile for the Lenovo ThinkPad T14s Gen 6

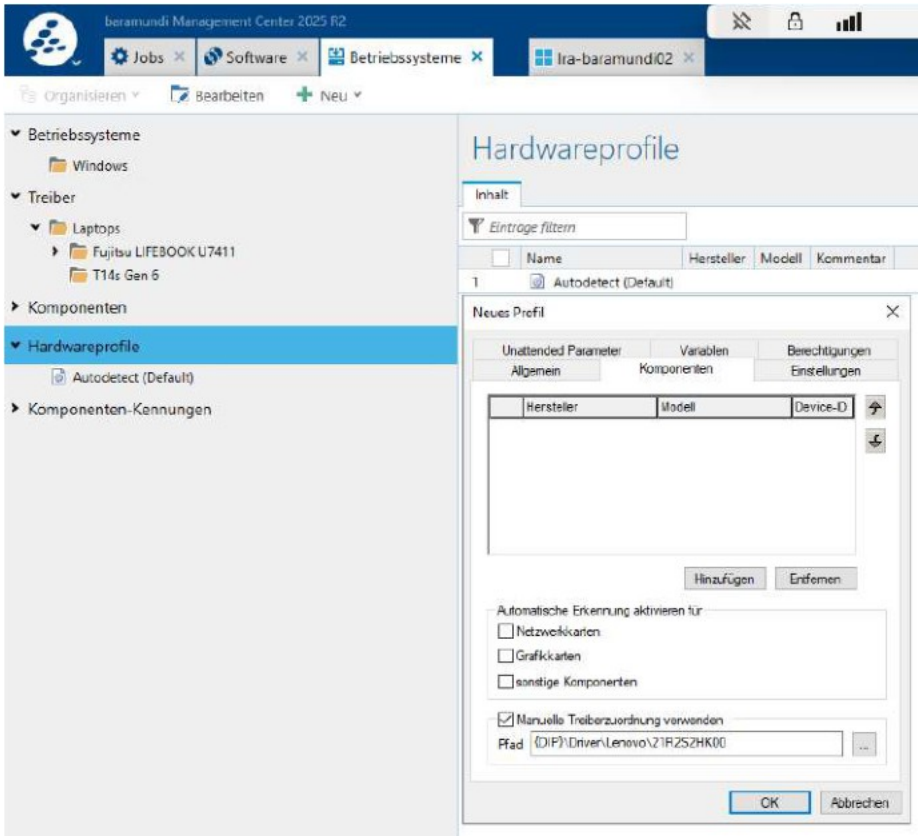


Figure 36: Manual assignment of the driver path to the Lenovo hardware profile

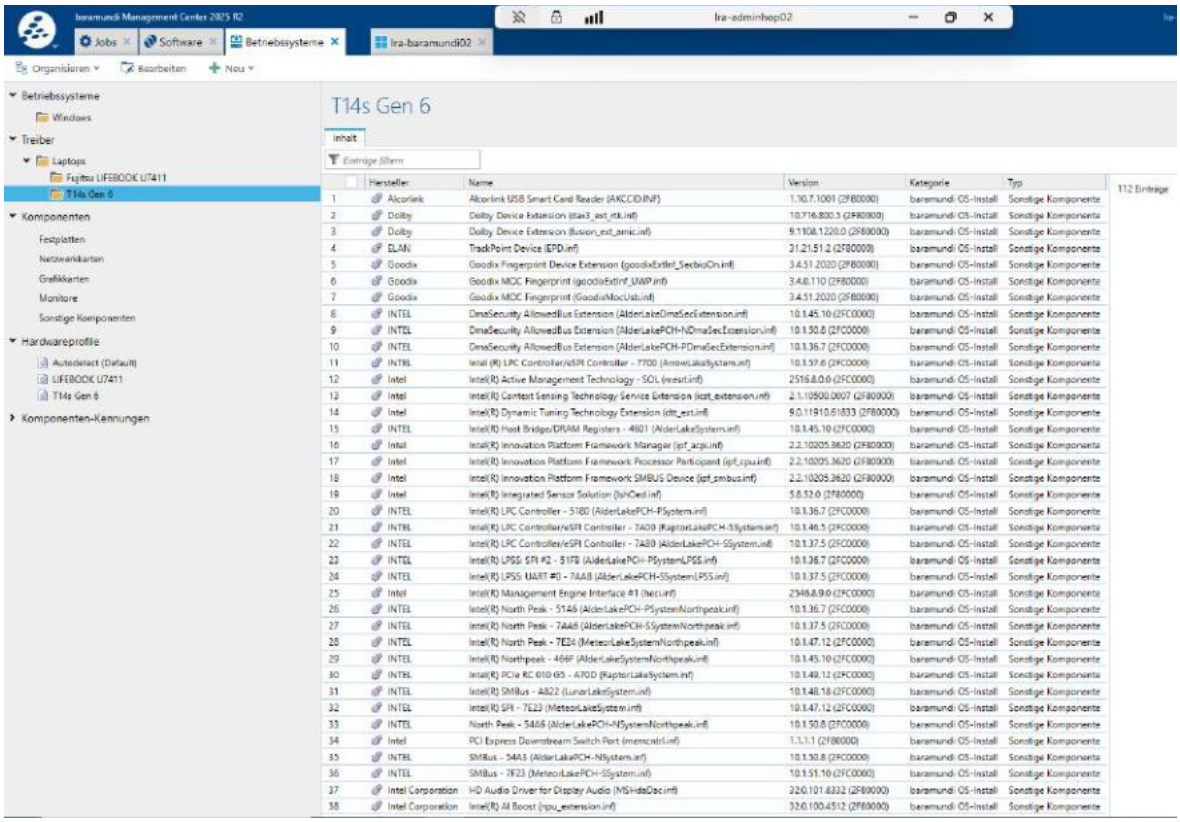


Figure 37: Imported Windows 11 drivers within the Lenovo hardware profile

The typical post-installation workflow therefore consists of importing drivers, creating a hardware or driver profile and assigning it to the appropriate hardware model or OS installation job. This avoids the need to integrate drivers indiscriminately into the operating system image.

4. Test rollout and troubleshooting

4.1 Test rollout and fault analysis

The prepared Testgruppe_UEM_Evaluation was used for the pilot. The jobs for operating system installation, baseline software deployment and inventory collection were assigned to this group. This allowed the reference client to be provisioned within the defined test scope and its functionality subsequently assessed. The pilot therefore leads directly into the test rollout and fault analysis.

The prepared baramundi deployment was tested with a reference client during the test rollout. Initially, the client contacted the existing Empirum server rather than the baramundi server. As this behaviour persisted despite a checked configuration and the explicit exclusion of the Empirum server from VLAN 33, the cause could not initially be isolated conclusively within the baramundi system.

Troubleshooting was carried out jointly with the IT specialists responsible for the network segment. DHCP/PXE assignments, firewall rules and the relevant technical documentation were checked, among other things. As the Empirum server could not be taken out of service without coordination and without risking

production operations, a four-working-day maintenance window had to be awaited for the cross-check. Once the Empirum server had been briefly disconnected from the network, interference from Empirum ceased.

The client then reached the PXE phase, but the subsequent startup process did not successfully enter the baramundi boot environment. The decisive point of failure was therefore not the underlying DHCP/PXE configuration but the subsequent WinPE initialisation. The WinPE boot environment did not contain a compatible network driver for the test client's hardware. Consequently, the network adapter could not be initialised correctly after WinPE started, even though the PXE process had been initiated.

After the WinPE boot image was recreated and the appropriate network drivers integrated, PXE booting through baramundi succeeded. The procedure follows the WinPE boot environment creation process described in section 3.8.

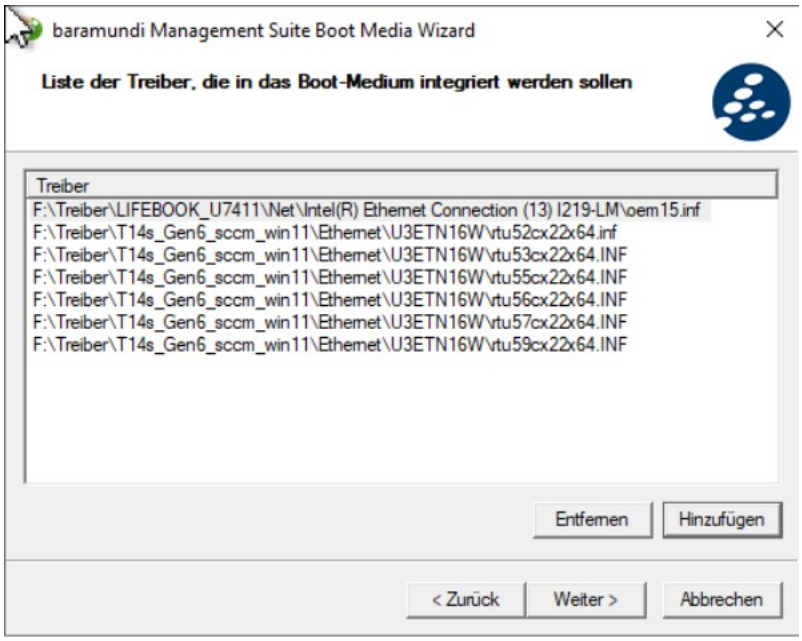


Figure 38: Integration of network drivers into the WinPE boot environment using the Boot Media Wizard

The installation then progressed further, but repeatedly failed during the Windows Setup phase.

Figure 39: Windows Setup — failure of the unattended installation on the test client

The Windows installation setup logs were then examined to isolate the problem:

- dir C:\Windows\Panther
- type C:\Windows\Panther\setuperr.log
- type C:\Windows\Panther\setupact.log

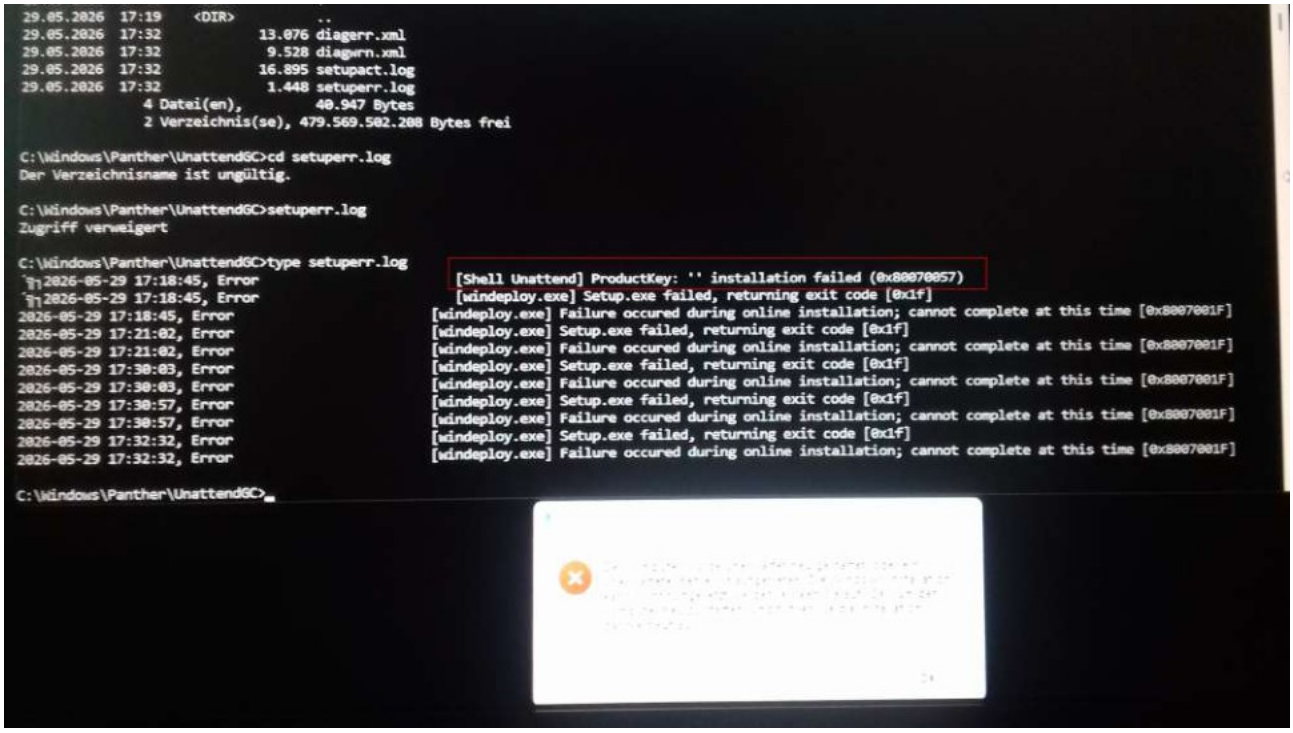


Figure 40: Examination of the setup logs indicating the missing ProductKey

Analysis of the logs revealed the error message ‘ProductKey: " installation failed (0x80070057)’. The cause was a missing product key in the unattended Windows installation. A generic Microsoft installation key for

the Windows edition in use was then entered in baramundi. This key is not used for activation; it permits installation of the relevant edition and allows setup to complete correctly.

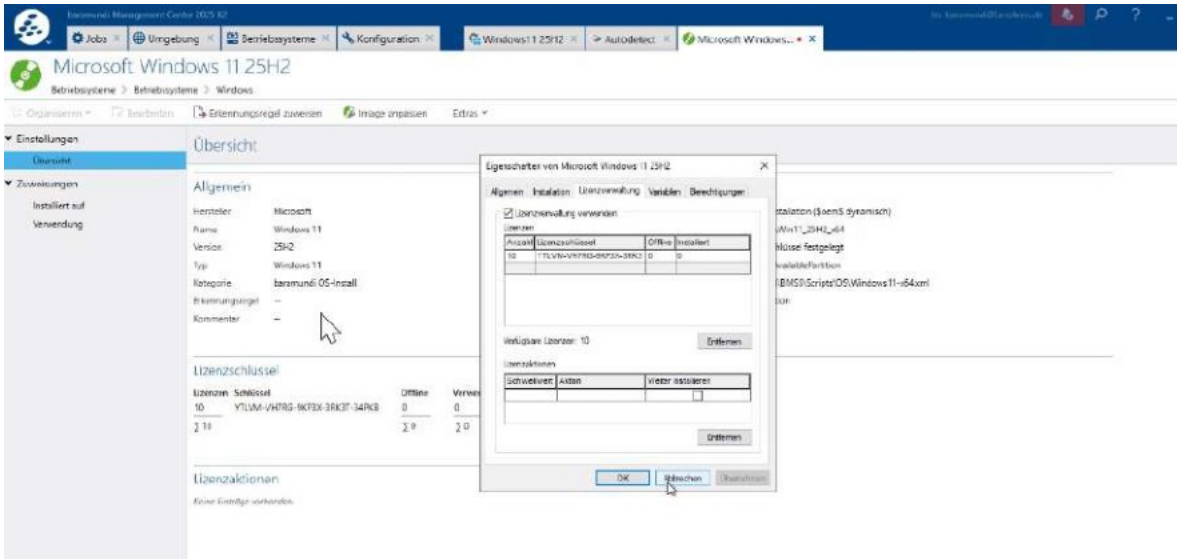


Figure 41: Entry of the generic installation key in the Windows installation object

Following this correction, the four clients tested within the available time window were successfully provisioned. After the operating system rollout, the baramundi Management Agent was deployed to the client. This agent is required for subsequent baramundi jobs, such as software installations, inventory collection and status reporting, to run on the target system and report back to the Management Suite. Assignment through the Test_UEM_Evaluation group worked without further functional faults. The operating system rollout, baseline client software deployment and inventory jobs all completed verifiably in the subsequent test run.

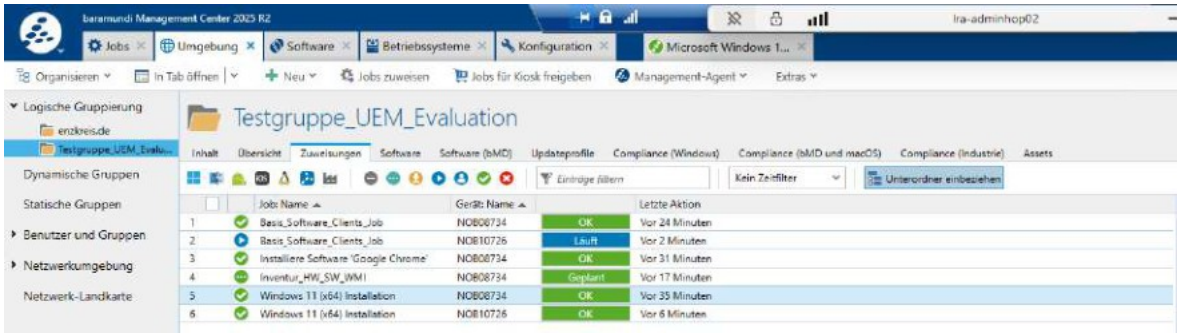


Figure 42: Successfully completed rollout in the test run, including baseline software jobs and the inventory job

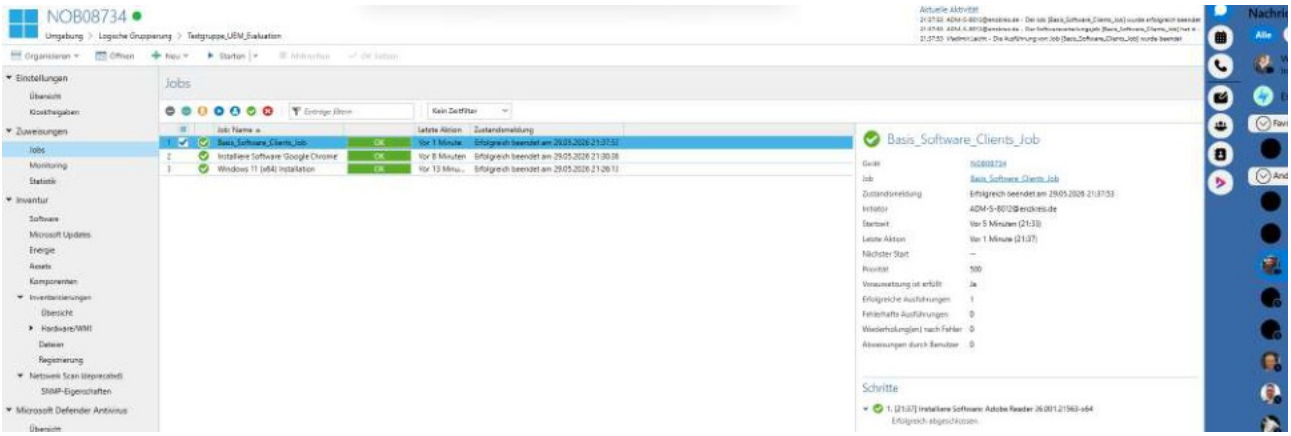


Figure 43: Successfully completed rollout no. 2 in the test run, including baseline software jobs and the inventory job

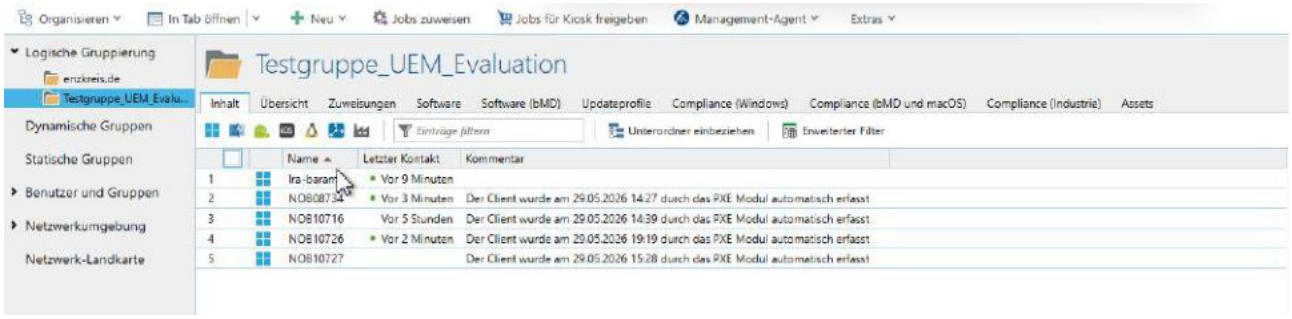


Figure 44: Successful client registration in the test group

5. Project completion

5.1 Planned versus actual results

Comparing the planned objectives with the results achieved shows that the project scope was essentially implemented in full. The on-premises bMS server was successfully provisioned and connected to the MSSQL database. Server–client communication is encrypted using TLS on port 443, bringing the connection into line with customary security requirements.

The core functional objectives were also achieved. Automated Windows 11 PXE deployment was configured so that installation of the target systems could be initiated reproducibly. Deployment of the specified standard applications through bDeploy broadly proceeded as planned. The five reference clients envisaged in the project proposal were not all completed owing to time constraints; four clients were fully provisioned. After the faults described in the test rollout had been resolved, the four provisioning runs actually performed completed without further functional problems. The success rate for these four completed test runs was therefore 100%, as evidenced by the results screenshots in section 4.1. This did not invalidate the pilot rollout’s functional objective, since operating system installation, baseline software deployment and inventory collection were all successfully demonstrated.

5.2 Time management

The project work was carried out in separate time windows. Net project time remained limited to 40 hours. Waiting periods, approvals and work performed by other departments were not counted as the candidate's project time; only time spent directly on planning, implementation, testing, research and documentation was included.

As work in the PoC subnet was not permitted to disrupt ongoing operations, production processes took priority. Periods without immediate technical progress were therefore used for research, information gathering, checking technical dependencies and preparing documentation alongside the project, provided that these tasks did not overlap with other duties and appointments the candidate had at times.

Time window	Main activities	Net time
08.05.	Foundations for server provisioning in vSphere and preparation of the technical starting point	1.6 h
11.05.–12.05.	Installation and initial configuration of Windows Server, baramundi Management Suite, database connection and DIP	12.0 h
13.05.–19.05.	Isolation of Empirum interference, research, information gathering and preparation of the documentation framework during interrupted project phases; including 2.4 h of documentation preparation and 5.0 h of contingency/research time	7.4 h
20.05.	Agreement on the next steps and coordination of the required maintenance window as contingency/coordination time	1.0 h
21.05.–28.05.	Contingency period for research, checking technical dependencies and coordination while implementation remained restricted	2.0 h
29.05.	Maintenance window with the Empirum server switched off, final test rollout, driver adjustment, ProductKey correction and validation	8.0 h
31.05.	Completion, formal cross-checking and final correction of the project documentation	8.0 h
Total		40.0 h

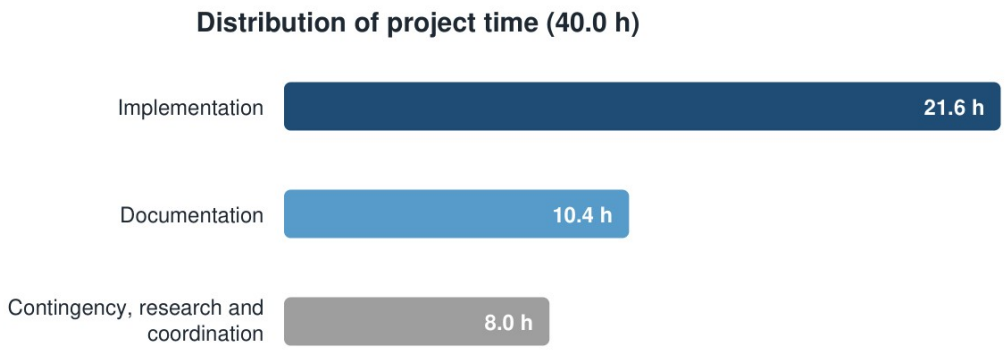


Figure 45: Distribution of project time by net effort

Implementation therefore accounted for 21.6 hours. Documentation comprised 2.4 hours of preparation alongside the project and 8.0 hours of final correction, totalling 10.4 hours. The remaining contingency, research and coordination activities accounted for 8.0 hours. The extended calendar period therefore did not result in the permitted project time being exceeded.

5.3 Conclusion and outlook

5.3.1 Conclusion

The migration of the endpoint management infrastructure from Matrix42 Empirum to the baramundi Management Suite was successfully completed within the planned timeframe. The central technical requirements were implemented without fundamental departures from the project plan. These included, in particular, stabilised Windows 11 deployment through UEFI PXE, provision of standard software through bDeploy and integration of the basic management and patch management functions.

Following troubleshooting, the relevant deployments were successful in the test environment. This does not mean that every individual operation is inherently risk-free; within the defined project scope, however, the outcome was stable and verifiable.

5.3.2 Outlook

The next phase is intended to extend the baramundi Management Agent gradually across the authority's remaining client estate. A later project stage should also assess whether the Mobile Device Management module could usefully provide consistent, centralised management of mobile endpoints.

The economic assessment indicates that reduced processing times and improved standardisation could have a positive effect on ongoing operating costs. Responsibility for packaging and software deployment can also

be shared among more employees. This reduces dependence on individual specialists while increasing operational flexibility.

5.4 Lessons learned / reflections

Alongside the technical findings, the project provided greater insight into working processes within a professional IT environment. Implementation showed that informed, regular coordination between the departments involved is an essential prerequisite for resolving technical issues. Responsibilities, prerequisites and changes should be clarified as early as possible, particularly for network segmentation, DHCP/PXE dependencies, service accounts and server provisioning.

It also became clear that patience and a systematic approach are crucial when analysing faults. Not every problem could immediately be attributed to a single application component, process or configuration. Causes could only be isolated and resolved verifiably by checking the server configuration, network paths, permissions and baramundi settings step by step.

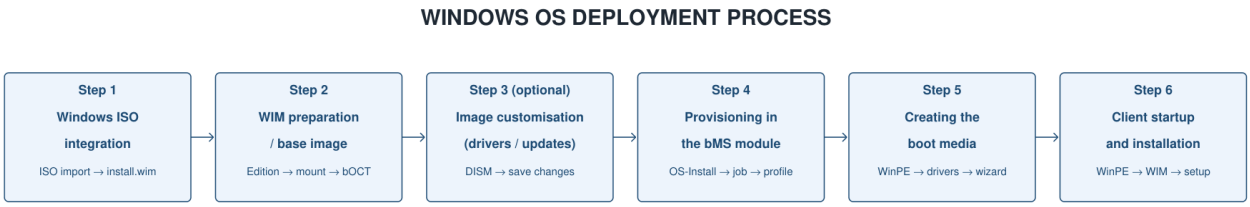
Practical knowledge and skills were expanded particularly in the following areas:

- Virtualisation environments
  - Provisioning and checking the server VM in vSphere
  - Relationship between VM configuration, network connectivity and operating system installation
- Initial Windows Server configuration
  - Configuring a static IP address
  - Configuring the server’s DNS connection
  - Preparing the server to run the baramundi Management Suite
- Active Directory and permissions
  - Working with administration groups
  - Setting up service accounts and local permissions
  - Importance of a clear permissions model for installation, operation and troubleshooting
- Network and deployment dependencies
  - Coordinating DHCP, PXE and TFTP
  - Accounting for VLAN and DMZ segmentation
  - Connecting and testing clients
  - Recognising that OS deployment cannot be considered solely in terms of the management server
- baramundi architecture
  - Interaction between Management Server, DIP/baraDIP, bMA, bDeploy, patch management and inventory collection
  - Understanding the architecture through practical configuration and pilot operation

- Structured fault analysis
  - Analysis of the IPv4/IPv6 or vSphere network problem
  - Recognising that fault symptoms often originate somewhere other than the point at which they become visible

Future projects would therefore benefit from even earlier coordination of the technical prerequisites and consistent documentation of dependencies.

During preparation of the documentation, the technical processes triggered by the respective GUI actions were examined in greater depth. The following overview summarises the steps and background processes in this OS deployment lifecycle:



Step 1: Integrating the Windows ISO into baramundi

- Importing the Windows ISO: reading the original ISO media [Windows ISO: standard operating system image file. Keywords: source media, ISO image].
- Extracting install.wim: unpacking the central file container [install.wim: Microsoft's file-based image format for system files. Keywords: WIM[22], image file].
- Image analysis: reading the available Windows editions for selection.

Step 2: WIM preparation / base image

- Edition selection: selecting the target version, for example Windows 11 Enterprise LTSC [Windows edition: operating system variant for enterprise use. Keywords: LTSC, Windows version].
- Preparation: mounting the image for offline customisation [Mounting: attaching a virtual image for offline modification. Keywords: DISM[21], offline mount].
- Handover to bOCT[10]: providing the base WIM in the OS Customization Tool [OS Customization Tool: baramundi configuration tool for image customisation. Keywords: bOCT, image preparation].

Step 3: Image customisation (optional)

- Driver integration: offline injection of hardware-level INF drivers [Hardware drivers: low-level drivers for chipsets, networking and storage. Keywords: INF integration, DISM injection].

- Integrating updates: incorporating cumulative security updates [Windows updates: offline patch integration to reduce subsequent patching effort. Keywords: slipstreaming, patch integration].
- Saving: unmounting and permanently saving the changes (commit) [Commit: saving the changes made to the WIM image. Keywords: Save-Changes, commit action].

Step 4: Provisioning in the baramundi OS-Install module

- Image import: loading the customised WIM image into the OS-Install module [baramundi OS-Install: baramundi module for automated operating system deployment. Keywords: OS deployment, bMS module].
- Job association: assigning the image to the relevant installation jobs [Deployment jobs: installation steps executed sequentially on the client. Keywords: bMS job, task chain].
- Target group definition: linking the job to the target hardware profiles [Target groups / profiles: conditions for dynamic distribution to endpoints. Keywords: client group, profile assignment].

Step 5: Creating the boot media

- WinPE startup environment: creating a minimal boot operating system in RAM [WinPE: minimal operating system in memory for preparing setup. Keywords: preinstallation, RAM disk].
- PE driver integration: incorporating essential network and storage drivers [Boot drivers: NIC and controller drivers essential for WinPE startup. Keywords: NIC drivers, storage drivers].
- Wizard output: creating the startup media (PXE or USB media) using the Boot Media Wizard [Boot Media Wizard: wizard for creating bootable media. Keywords: bBMW, ISO output].

Step 6: Client startup and installation

- WinPE boot: starting the client in the WinPE environment via PXE or USB [WinPE boot: the client startup phase within the WinPE runtime environment. Keywords: boot phase, PE startup].
- Image retrieval: establishing the network connection and downloading the WIM image from the DIP.
- Setup execution: extracting the WIM image and performing a fully automated installation (Unattended Setup[12]) [Unattended Setup: unattended Windows installation using an answer file (autounattend.xml[23]). Keywords: autounattend.xml, silent install].

Appendix A: Glossary and sources

1 Matrix42 Empirum: endpoint management and software deployment solution replaced as the existing client management environment in the project. Source: Matrix42 documentation. [Return to the reference in the report]

2 baramundi Management Suite (bMS): unified endpoint management platform for centralised management of inventory collection, software deployment, patch management and operating system deployment. Source: baramundi product page. [Return to the reference in the report]

3 Distributed Installation Point (DIP) / baraDIP: baramundi component for providing and synchronising installation sources, software packages and deployment data. Source: baramundi system requirements. [Return to the reference in the report]

4 bConnect interface: HTTPS-based interface of the baramundi Management Suite for connecting external systems and components. Source: baramundi Interfaces. [Return to the reference in the report]

5 bDeploy: baramundi function for packaging and automatically deploying software to managed endpoints. Source: baramundi Management Suite. [Return to the reference in the report]

6 Boot Media Wizard: wizard for creating bootable WinPE media for PXE- or USB-based operating system deployment. Source: baramundi Help Center. [Return to the reference in the report]

7 PXE/TFTP: mechanisms for booting a client over the network and transferring the necessary boot files. Source: Microsoft PXE troubleshooting. [Return to the reference in the report]

8 baramundi Management Agent (bMA): client component of the baramundi environment used to control inventory collection, job execution and software deployment on endpoints. Source: baramundi Management Suite. [Return to the reference in the report]

9 bBT: transfer mechanism used in the baramundi environment for data access between the client, service account and DIP structure. Source: baramundi system requirements. [Return to the reference in the report]

10 bOCT / OS Customization Tool: baramundi tool for preparing and customising Windows images as part of operating system deployment. Source: baramundi Help Center. [Return to the reference in the report]

11 baramundi OS-Install: baramundi module for automated operating system installation and association of images with installation jobs. Source: baramundi Management Suite. [Return to the reference in the report]

12 Unattended Setup: unattended Windows installation in which settings are supplied automatically through an answer file. Source: Microsoft Unattended Setup Reference. [Return to the reference in the report]

13 ASP.NET Core Runtime: Microsoft runtime environment for running ASP.NET Core applications and web-based server components. Source: Microsoft .NET installation. [Return to the reference in the report]

14 Software dependencies: technical prerequisites such as runtime environments, libraries or services required to install and operate an application. Source: project context. [Return to the reference in the report]

15 ErrorLevel 3010: return-code evaluation in scripts; code 3010 typically indicates a successful installation requiring a restart. Source: Microsoft Windows Installer Error Messages. [Return to the reference in the report]

16 MSW and patch sources: baramundi sources for Managed Software and patch data provided or processed through the DIP. Source: baramundi Managed Software. [Return to the reference in the report]

17 BDS files: baramundi data or control files read and processed by the server services. Source: baramundi project context. [Return to the reference in the report]

18 DIP$: administrative Windows share of the Distributed Installation Point for server-side access to installation and package data. Source: baramundi system requirements. [Return to the reference in the report]

19 bCenter connection: connection of the baramundi Management Console or server components to the bMS database and management environment. Source: baramundi Help Center. [Return to the reference in the report]

20 Windows ADK: Microsoft toolset for Windows deployment, image customisation and creation of WinPE environments. Source: Microsoft Learn. [Return to the reference in the report]

21 DISM: Deployment Image Servicing and Management; Microsoft tool for servicing, checking and modifying Windows images. Source: Microsoft Learn. [Return to the reference in the report]

22 WIM image: file-based Windows image format for storing and distributing Windows installation or boot images. Source: Microsoft DISM documentation. [Return to the reference in the report]

23 autounattend.xml: answer file for unattended Windows installation that supplies setup options automatically. Source: Microsoft Unattended Setup. [Return to the reference in the report]

24 DHCP options 66 and 67: DHCP boot parameters: option 66 identifies the boot server, and option 67 specifies the startup file to load. Source: Microsoft PXE troubleshooting. [Return to the reference in the report]

25 bootx64.efi: UEFI startup file for 64-bit systems that can be loaded as a bootloader during network startup. Source: Microsoft PXE troubleshooting. [Return to the reference in the report]